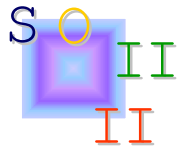
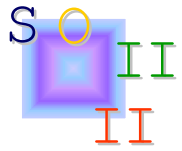
Tema 3. Concurrencia entre procesos



Índice

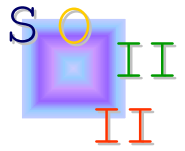
- Procesamiento concurrente
- El problema de la sección crítica
- Semáforos
- Mutex y variables de condición
- Señales
- Paso de mensajes
- Monitores
- Mecanismos de concurrencia en sistemas Unix
- Mecanismos de concurrencia en Windows NT/2000
- Mecanismos de concurrencia en Minix

Tema 3. Concurrencia entre procesos



Bibliografía

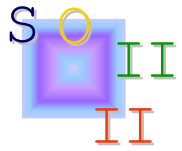
- J. Carretero et al. *Sistemas Operativos: Una Visión Aplicada.* McGraw-Hill. 2001. Capítulo 5.
- W. Stallings. *Sistemas Operativos*. Prenctice-Hall. 2001. Capítulos 5 y 6.
- A.S. Tanenbaum, A.S. Woodnull. *Operating Systems. Design and Implementation*. Prentice-Hall International, 2a. edición. 1996.
- H. Custer. *Inside Windows NT.* Microsoft Press. 1993. Capítulos 4 y 5.



Bibliografía (cont.)

- F. Pérez, J. Carreter, F. García. *Problemas de Sistemas Operativos: De la Base al Diseño.* McGraw-Hill. 2003. Capítulo 5.
- M. A. Castaño, J. Echagüe, R. Mayo, C. Pérez. *Problemas de Sistemas Operativos.* Col.lecciò “Materials”. Servicio de Publicaciones de la UJI, num. 109. 2000. Capítulo 3.
- K.A. Robbins, S. Robbins. *Unix Programación Práctica*. Prenctice-Hall. 1997. Capítulos 5, 8, 9 y 10.

Tema 3. Concurrencia entre procesos

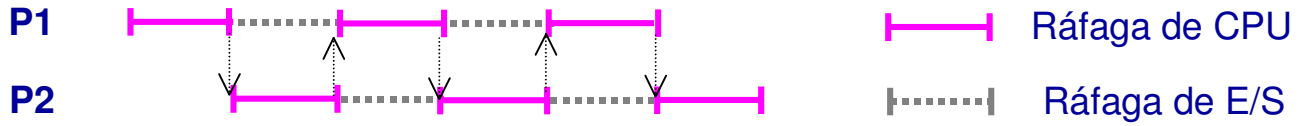


Índice

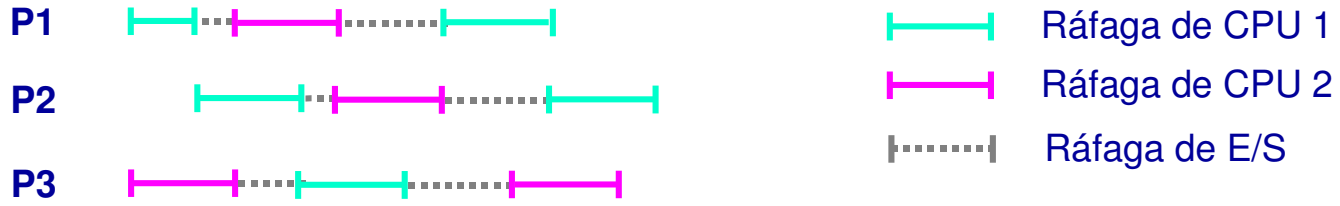
- Procesamiento concurrente
- El problema de la sección crítica
- Semáforos
- Mutex y variables de condición
- Señales
- Paso de mensajes
- Monitores
- Mecanismos de concurrencia en sistemas Unix
- Mecanismos de concurrencia en Windows NT/2000
- Mecanismos de concurrencia en Minix

Procesamiento concurrente

- **Modelos de computadora en los que se puede dar:**
  - *Multiprogramación en un único procesador*
    - Procesamiento concurrente: base de los SOs multiprogramados

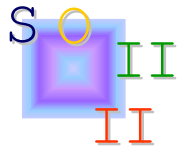


  - *Multiprocesador*
    - Los procesos concurrentes no sólo pueden intercalar su ejecución sino también superponerla
    - Existe verdadera ejecución simultánea de procesos



  - *Multicomputador* (proceso distribuido)

Procesamiento concurrente



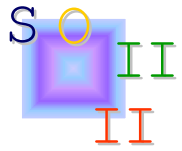
- **Razones de la ejecución concurrente:**
  - Compartir recursos físicos
  - Compartir recursos lógicos

  } → Mejor aprovechamiento

  - Acelerar los cálculos
  - Modularidad
  - Comodidad

  } → Facilitar programación

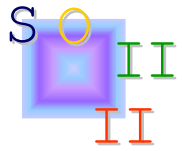
Procesamiento concurrente



- **Tipos de procesos concurrentes:**
  - **Procesos independientes** no pueden afectar o ser afectados por la ejecución de otro proceso

    Procesos cooperantes que comparten datos pueden generar inconsistencia en esos datos
  - Interacción entre procesos
    - Compiten por recursos
    - Comparten recursos
    - Ejecución sincronizada
  - Se necesita:
    - Mecanismos de **sincronización y comunicación entre procesos**
    - Ejecución ordenada para conseguir datos consistentes

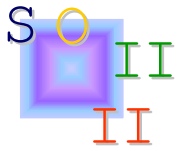
Tema 3. Concurrencia entre procesos



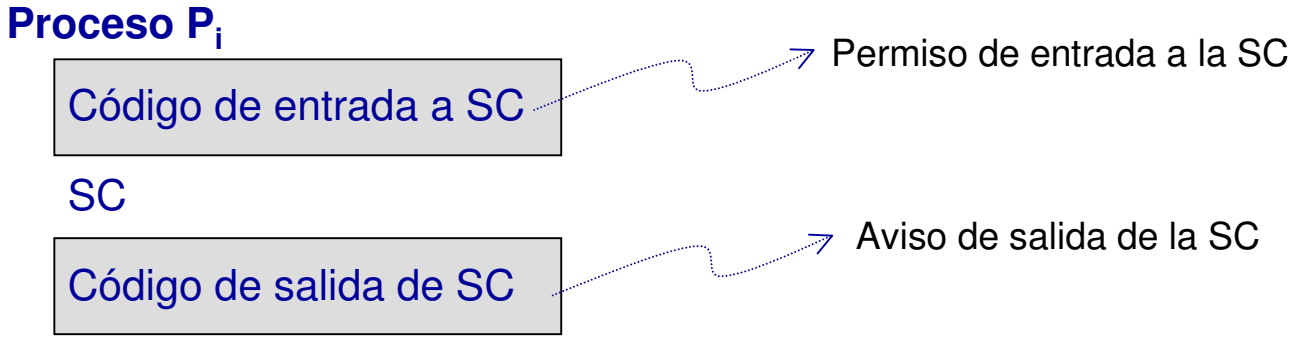
Índice

- Procesamiento concurrente
- El problema de la sección crítica
- Semáforos
- Mutex y variables de condición
- Señales
- Paso de mensajes
- Monitores
- Mecanismos de concurrencia en sistemas Unix
- Mecanismos de concurrencia en Windows NT/2000
- Mecanismos de concurrencia en Minix

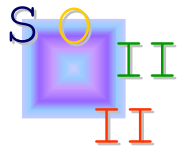
El problema de la sección crítica



- **Planteamiento:**
  - **n** procesos **P_i i=1,..,n** compitiendo por usar ciertos datos compartidos
  - Cada proceso tiene un fragmento de código, llamado **sección crítica** (SC), en el que el proceso accede a los datos compartidos
- **Problema:**
  - Asegurar que cuando un proceso está ejecutando su sección crítica ningún otro proceso puede estar ejecutando su sección crítica
- **Solución:**
  - Añadir código adicional a los programas para acceder a y salir de la SC

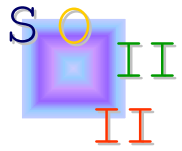


El problema de la sección crítica



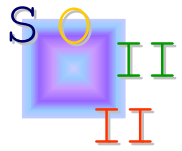
- **Requisitos que ha de cumplir una solución al problema de la SC:**
  - **Exclusión mutua**:
    - Sólo debe haber un proceso ejecutando la SC
  - **Progreso**:
    - Un proceso fuera de la SC no debe bloquear a otro que quiere entrar
  - **Espera limitada**:
    - Un proceso que quiere entrar en la SC no espera indefinidamente

El problema de la sección crítica



- **Herramientas de comunicación proporcionadas por el SO:**
  - Archivos
  - Tuberías
  - Variables en memoria compartida
  - Paso de mensajes
- **Herramientas de sincronización proporcionadas por el SO (o por el entorno de desarrollo):**
  - Señales
  - Tuberías
  - Semáforos
  - Mutex y variables condicionales
  - Monitores
  - Paso de mensajes

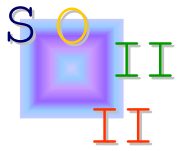
Tema 3. Concurrencia entre procesos



Índice

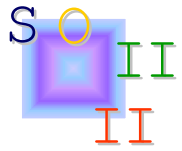
- Procesamiento concurrente
- El problema de la sección crítica
- Semáforos
- Mutex y variables de condición
- Señales
- Paso de mensajes
- Monitores
- Mecanismos de concurrencia en sistemas Unix
- Mecanismos de concurrencia en Windows NT/2000
- Mecanismos de concurrencia en Minix

Semáforos



- **¿Qué es un semáforo?**
  - Solución para el problema de la SC
  - Solución para sincronización entre procesos
- **Definición de semáforo:**
  - Estructura de datos que sólo soporta tres operaciones:
    - **Inicialización**, **espera** y **señal**
    - Operaciones atómicas

Semáforos



- **Definición de semáforo sin espera activa:**

```
tipo semaforo=registro
        int valor;
        lista_de_procesos_bloqueados_en_el_semaforo L;
    end;

variable semaforo S;

wait (S):                                   Pedir recurso semáforo
    S.valor:=S.valor - 1;
    Si (S.valor < 0)
        entonces
                añadir a S.L el  proceso que invoca la función;
                bloquear este proceso;
     fin_si;
```

Semáforos

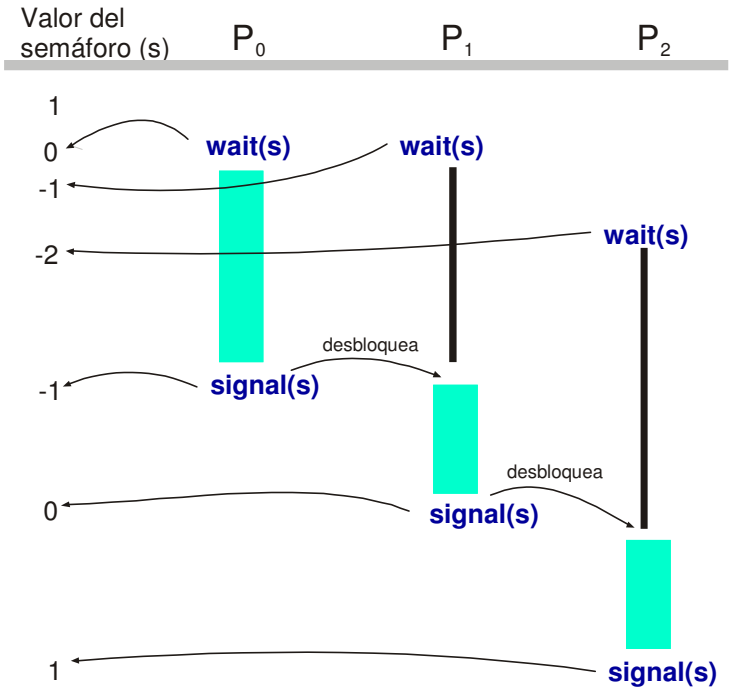
- **Definición de semáforo sin espera activa (cont.):**

Liberar recurso semáforo

```
signal (S):
    S.valor:=S.valor + 1;
    Si (S.valor ≤ 0)
        entonces
                extraer un proceso P de S.L;
                desbloquear P e insertarlo en lista de procesos preparados;
    fin_si;
```

Semáforos

- **Problema de la SC para *n* procesos con semáforos:**



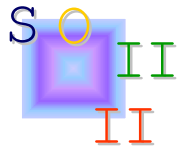
Datos compartidos:
variable semaforo S;
sem_init (S,1);

Proceso P_i
wait (S);
SC;
signal (S);

Ejecutando código de la sección crítica

Proceso bloqueado en el semáforo

Semáforos



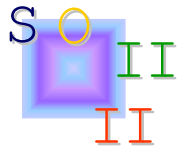
- **Semáforos como herramienta de sincronización entre procesos:**
  - Ejemplo:
    - Ejecución de la instrucción **B** en $\mathbf{P_j}$ después de ejecutar la instrucción **A** en $\mathbf{P_i}$.

Datos compartidos:

variable semaforo sinc;
sem_init (sinc,0);

Proceso P_i	**Proceso P_j**
...	...
instrucción A;	wait (sinc);
signal (sinc);	instrucción B;
...	...

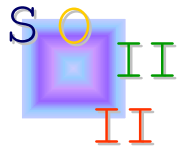
Servicios POSIX sobre semáforos



- **Identificación de un semáforo en POSIX:**
  - Variable del tipo `sem_t`
- **Tipos de semáforos en POSIX:**
  - Semáforos sin nombre:
    - Sincronizan hilos de un mismo proceso o procesos que heredan el semáforo a través de `fork`
  - Semáforos con nombre :
    - Sincronizan procesos no heredados a través de `fork`

Diferencia entre semáforos con y sin nombre: análoga a la que existe entre tuberías con y sin nombre.

Servicios POSIX sobre semáforos sin nombre



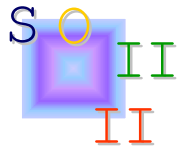
- **Funciones sobre semáforos en POSIX:**

```
int sem_init    (sem_t *sem, int shared, int val);
int sem_destroy (sem_t *sem);
int sem_wait    (sem_t *sem);
int sem_post    (sem_t *sem);
```

devuelven:

- Si todo ha ido bien: 0
- Si error: -1

Creación de semáforos sin nombre



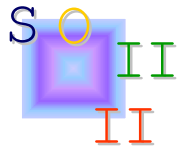
- **Sintaxis:**

```
int sem_init (sem_t *sem, int shared, int val);
```

- **Descripción:**
  - Crea un semáforo identificado a través de `sem` y le asigna el valor inicial `val`
  - Si `val=0` → lo usarán hilos del proceso que lo inicializa

    Si `val≠0` → lo usarán procesos que lo hereden mediante `fork`

Destrucción de semáforos sin nombre

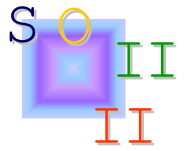


- **Sintaxis:**

```
int sem_destroy (sem_t *sem);
```

- **Descripción:**
  - Destruye un semáforo identificado a través de `sem`

Operaciones *sem_wait* y *sem_post*



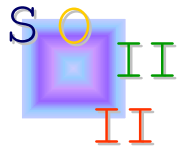
- **Operación `wait` sobre un semáforo POSIX:**
  - Sintaxis:

```
int sem_wait (sem_t *sem);
```

- **Operación `signal` sobre un semáforo POSIX :**
  - Sintaxis:

```
int sem_post (sem_t *sem);
```

Servicios POSIX sobre semáforos



- **Ejemplo 1: Acceso a SC con semáforos**

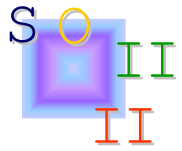
```
#include <pthread.h>
#include <stdio.h>
#include <unistd.h>
#include <stdlib.h>
#include <time.h>
#include <semaphore.h>

int x=0;
sem_t semaforo;

void *fhilo1(void *arg)
{ int i;
  for (i=0; i<3; i++) {
     sem_wait(&semaforo);
     x=x+1;
     sem_post(&semaforo);
     printf ("Suma 1\n");
     sleep (random()%3);
  }
  pthread_exit (NULL);
}
```

```
void *fhilo2(void *arg)
{ int i;
  for (i=0; i<3; i++) {
     sem_wait(&semaforo);
     x=x-1;
     sem_post(&semaforo);
     printf ("Resta 1\n");
     sleep (random()%3);
   }
   pthread_exit (NULL);
}
```

Servicios POSIX para gestión de hilos



- **Ejemplo 1 (cont.):**

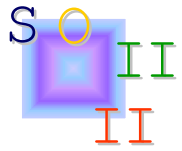
```
main()
{ pthread_t hilo1, hilo2;
  time_t t;
  srandom (time(&t);
  printf ("Valor inicial de x: %d \n",x);
  sem_init (&semaforo,0,1);

  pthread_create(&hilo1, NULL,  fhilo1, NULL);
  pthread_create(&hilo2, NULL,  fhilo2, NULL);

  pthread_join(hilo1,NULL);
  pthread_join(hilo2,NULL);
  sem_destroy (&semaforo);

  printf("Valor final de x: %d \n",x);
  exit(0);
}
```

Servicios POSIX sobre semáforos



- **Ejemplo 2: Sincronización con semáforos**

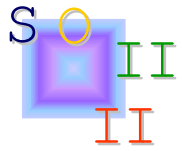
```
#include <pthread.h>
#include <stdio.h>
#include <unistd.h>
#include <stdlib.h>
#include <time.h>
#include <semaphore.h>
sem_t semaforo;

void *Escribe (char c, int nc)
{ int i;
  for (i=0; i<nc; i++) {
     printf ("%c\n",c);
     sleep (random()%3);
  }
}
```

```
void *fhilo1 (void *p)
{   Escribe ('A',3);
    sem_wait (&semaforo);
    Escribe ('C',2);
    pthread_exit (NULL);
 }

 void *fhilo2 (void *p)
 {  Escribe ('B',5);
    sem_post (&semaforo);
    pthread_exit (NULL);
 }
```

Servicios POSIX para gestión de hilos



- **Ejemplo 2 (cont.):**

```
main()
{ pthread_t hilo1, hilo2;
  time_t t;
  srandom (time(&t));
  sem_init (&semaforo,0,0);

  pthread_create(&hilo1, NULL,  fhilo1, NULL);
  pthread_create(&hilo2, NULL,  fhilo2, NULL);

  pthread_join(hilo1,NULL);
  pthread_join(hilo2,NULL);
  sem_destroy (&semaforo);

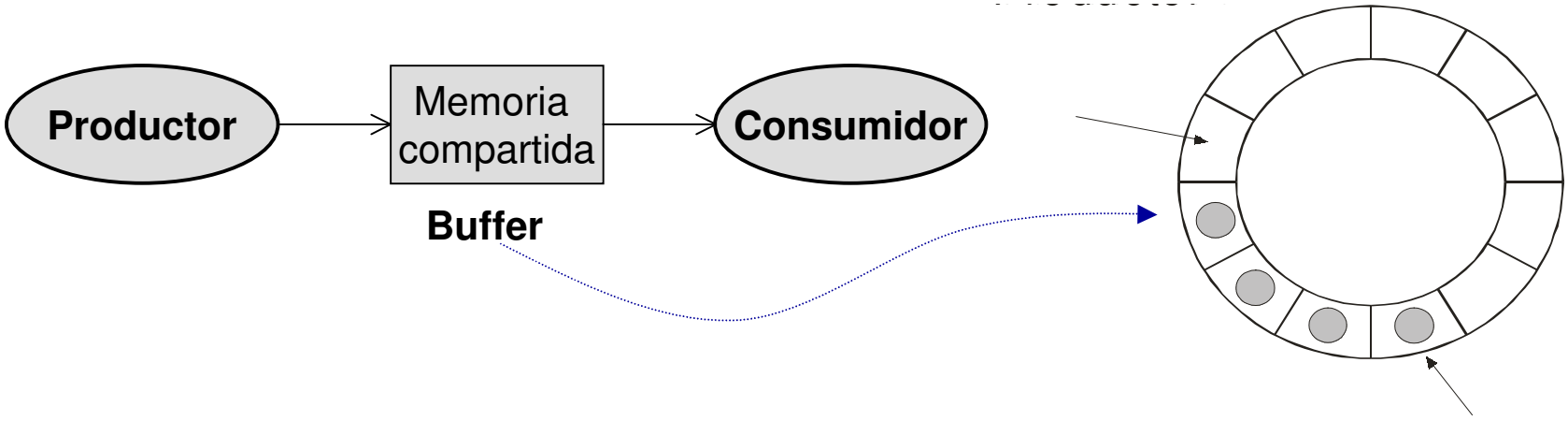
  exit(0);
}
```

¿Resultado de la ejecución?

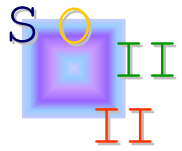
Servicios POSIX sobre semáforos

- **Ejemplo 3: El problema del productor-consumidor con buffer limitado (circular):**
  - Planteamiento:
    - El proceso productor produce información y la almacena en un buffer
    - El proceso consumidor accede al buffer y consume la información
    - El productor y el consumidor comparten variables → **Acceso a SC**
    - El productor no puede acceder al buffer si está lleno
    - El consumidor no puede acceder al buffer si está vacío

    → **Sincronización**



Servicios POSIX sobre semáforos



- **Ejemplo 3 (cont.):**

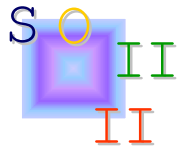
```
#include <pthread.h>
#include <stdio.h>
#include <unistd.h>
#include <stdlib.h>
#include <time.h>
#include <semaphore.h>

#define MAX_BUFFER          1024
#define DATOS_A_PRODUCIR  100000
sem_t elementos;
sem_t huecos;

int buffer[MAX_BUFFER];
```

```
/* Tamaño del buffer */
/* Datos a producir */
/* Elementos en el buffer */
/* Huecos en el buffer */
/* Buffer común */
```

Servicios POSIX sobre semáforos

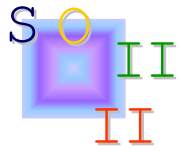


- **Ejemplo 3 (cont.):**

```
void *Productor(void)                          /* Código del Productor */
{   int pos = 0;                               /* Posición dentro del buffer */
    int dato;                                  /* Dato a producir */
    int i;

    for(i=0; i < DATOS A PRODUCIR; i++ )    {
        dato = i;                              /* Producir dato */
        printf ("Producido %d\n",dato);
        sem_wait(&huecos);                     /* Un hueco menos */
        buffer[pos] = i;
        pos = (pos + 1) % MAX BUFFER;
        sem_post(&elementos);                  /* Un elemento más */
        printf ("Producido %d\n",dato);
        sleep (random()%3);
    }
    pthread exit(0);
}
```

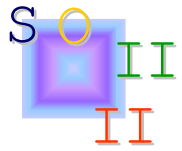
Servicios POSIX sobre semáforos



- **Ejemplo 3 (cont.):**

```
void *Consumidor(void)                          /* Código del Consumidor */
{   int pos = 0;
    int dato;
    int i;
    for(i=0; i < DATOS A PRODUCIR; i++ ) {
        sem_wait(&elementos);                   /* Un elemento menos */
        dato = buffer[pos];
        pos = (pos + 1) % MAX BUFFER;
        sem_post(&huecos);                      /* Un hueco más */
        printf ("Consumido %d\n",dato);         /* Cosumir dato */
        sleep (random()%3);
    }
    pthread exit(0);
}
```

Servicios POSIX sobre semáforos



- **Ejemplo 3 (cont.):**

```
void main(void)
{   pthread_t hilo1, hilo2;
    time_t t;
    srandom (time(&t));
    sem_init (&elementos, 0, 0);
    sem_init (&huecos, 0, MAX_BUFFER);

    pthread_create(&hilo1, NULL, Productor, NULL);
    pthread_create(&hilo2, NULL, Consumidor, NULL);

    pthread_join(hilo1, NULL);
    pthread_join(hilo2, NULL);
    sem_destroy (&huecos);
    sem_destroy (&elementos);

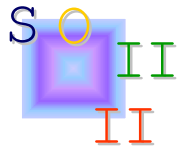
    exit(0);
}
```

¿Se accede a la SC en exclusión mutua?

Si hubiese 2 hilos productores y 2 consumidores, ¿sería correcto el código de las funciones `Productor` y `Consumidor`?

¿Cómo se daría prioridad a los procesos Consumidores?

Servicios POSIX sobre semáforos



- **Ejemplo 3 (cont.):**

```
#include <pthread.h>
#include <semaphore.h>
#include <stdio.h>

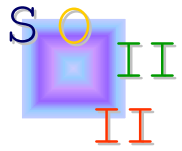
#define TAM_BUFFER          1024
#define DATOS_A_PRODUCIR 100000

sem_t elementos, huecos;
sem_t mutex1, mutex2;
sem_t prioridad;

int nconsumidores=0, nproductores=0;
int buffer[TAM_BUFFER];

int posprod=0, poscons=0;
```

Servicios POSIX sobre semáforos



- **Ejemplo 3 (cont.):**

```
void *f_productor(void *arg)
  { int i, dato;

    for (i=1; i<=DATOS_A_PRODUCIR; i++)
    { dato=i;
      sem_wait(&huecos);
      sem_wait(&prioridad);

      sem_wait(&mutex2);
      nproductores = nproductores +1;
      sem_post(&mutex2);

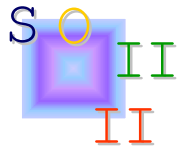
      sem_wait(&mutex1);
      buffer[posprod]=i;
      posprod=(posprod + 1) % TAM_BUFFER;
      printf("Producido %d \n", i);
      sem_post(&mutex1);

      sem_post(&elementos);

      sem_wait(&mutex2);
      if (nconsumidores == 0) sem_post(&prioridad);
      nproductores = nproductores +1;
      sem_post(&mutex2);

    }
    pthread_exit(0);
  }
```

Servicios POSIX sobre semáforos



- **Ejemplo 3 (cont.):**

```
void *f_consumidor(void *arg)
  { int i, dato;

    for (i=1; i<=DATOS_A_PRODUCIR; i++)
    { sem_wait(&elementos);

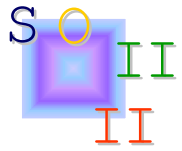
      sem_wait(&mutex2);
      nconsumidores=nconsumidores +1;
      sem_post(&mutex2);

      sem_wait(&mutex1);
      dato=buffer[poscons];
      poscons=(poscons + 1) % TAM_BUFFER;
      printf("Consumido %d \n", i);
      sem_post(&mutex1);

      sem_post(&huecos);

      sem_wait(&mutex2);
      nconsumidores=nconsumidores -1;
      if ((nconsumidores == 0) & (nproductores != 0)) sem_post(&prioridad);
      sem_post(&mutex2);
    }
    pthread_exit(0);
  }
```

Servicios POSIX sobre semáforos



- **Ejemplo 3 (cont.):**

```
main()
  {
    pthread_t productor1, productor2;
    pthread_t consumidor1, consumidor2;

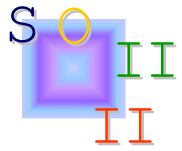
    sem_init(&elementos,0,0);   sem_init(&huecos,0,TAM_BUFFER);
    sem_init(&mutex1,0,1);      sem_init(&mutex2,0,1);
    sem_init(&prioridad,0,1);

    pthread_create(&productor1, NULL, f_productor, NULL);
    pthread_create(&productor2, NULL, f_productor, NULL);
    pthread_create(&consumidor1, NULL, f_consumidor, NULL);
    pthread_create(&consumidor2, NULL, f_consumidor, NULL);

    pthread_join(productor1,NULL);
    pthread_join(productor2,NULL);
    pthread_join(consumidor1,NULL);
    pthread_join(consumidor2,NULL);

    sem_destroy(&elementos); sem_destroy(&huecos);
    sem_destroy(&mutex1);    sem_destroy(&mutex2);
    sem_destroy(&prioridad);
    exit(0);
  }
```

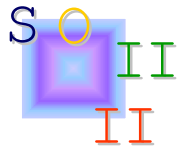
Tema 3. Concurrencia entre procesos



Índice

- Procesamiento concurrente
- El problema de la sección crítica
- Semáforos
- Mutex y variables de condición
- Señales
- Paso de mensajes
- Monitores
- Mecanismos de concurrencia en sistemas Unix
- Mecanismos de concurrencia en Windows NT/2000
- Mecanismos de concurrencia en Minix

Mutex



- **Definición de mutex:**
  - Mecanismo de sincronización (sencillo y eficiente) indicado para hilos
  - Se emplea para obtener acceso exclusivo a recursos compartidos y para "serializar" el acceso a la SC en exclusión mutua

    Sólo un hilo puede tener acceso simultáneamente al mutex
  - Semáforo binario con dos operaciones atómicas:
    - `lock(m)`:
      - Intenta bloquear el mutex `m`
      - Si el mutex ya está bloqueado el hilo se suspende
    - `unlock(m)`:
      - Desbloquea el mutex `m`
      - Si existen hilos bloqueados en el mutex se desbloquea a uno

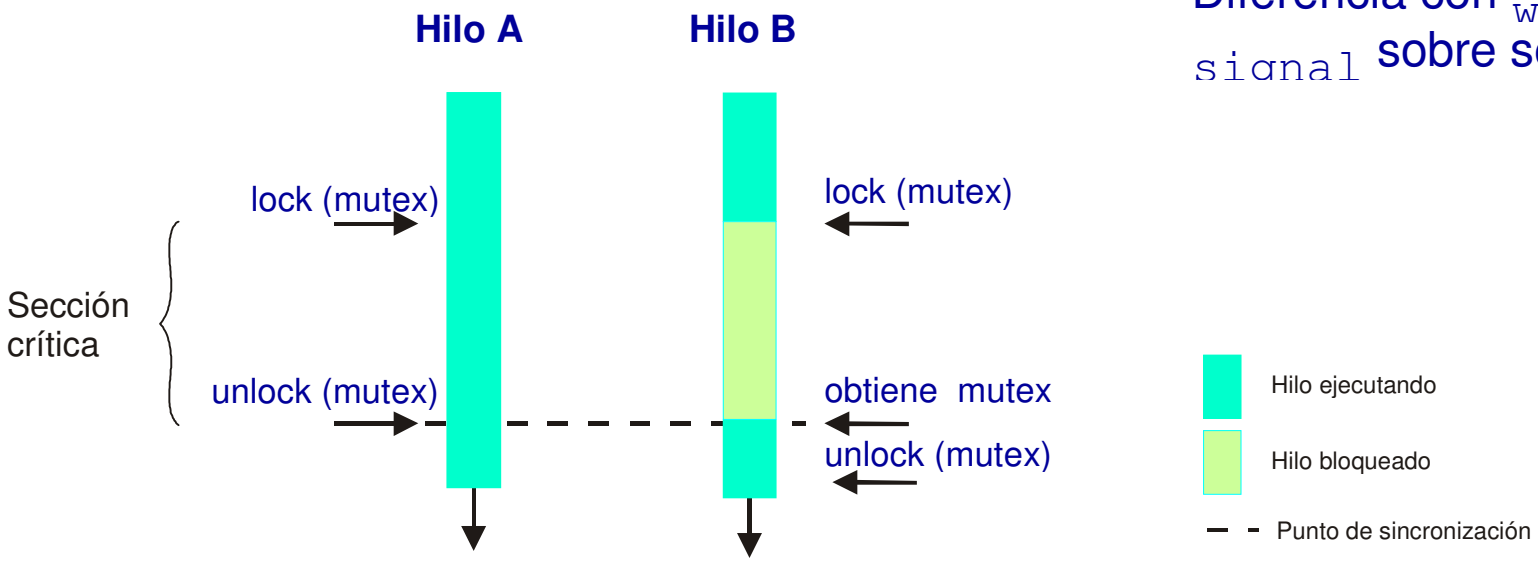
Secciones críticas con mutex

- **Utilización del mutex:**

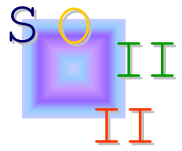
```
lock(m):          /* Entrada en la SC */
< seccion critica >
unlock(s):        /* Salida de la SC */
```

La operación `unlock` debe realizarla el hilo que ejecutó `lock`

Diferencia con `wait` y `signal` sobre semáforos

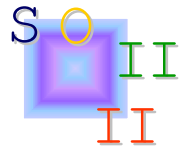


Variables condicionales



- **Definición de variable condicional:**
  - Variable de sincronización asociada a un mutex
  - Se usa entre `lock` y `unlock`
  - Dos operaciones atómicas asociadas:
    - **`wait (condition, mutex)`:**
      - ♣ Bloquea al hilo que la ejecuta y le expulsa del mutex
    - **`signal (condition, mutex)`:**
      - ♣ Desbloquea a uno o varios procesos suspendidos en la variable condicional `condition`
      - ♣ El proceso que se despierta compite de nuevo por el mutex

Uso de mutex y variables condicionales



Hilo A

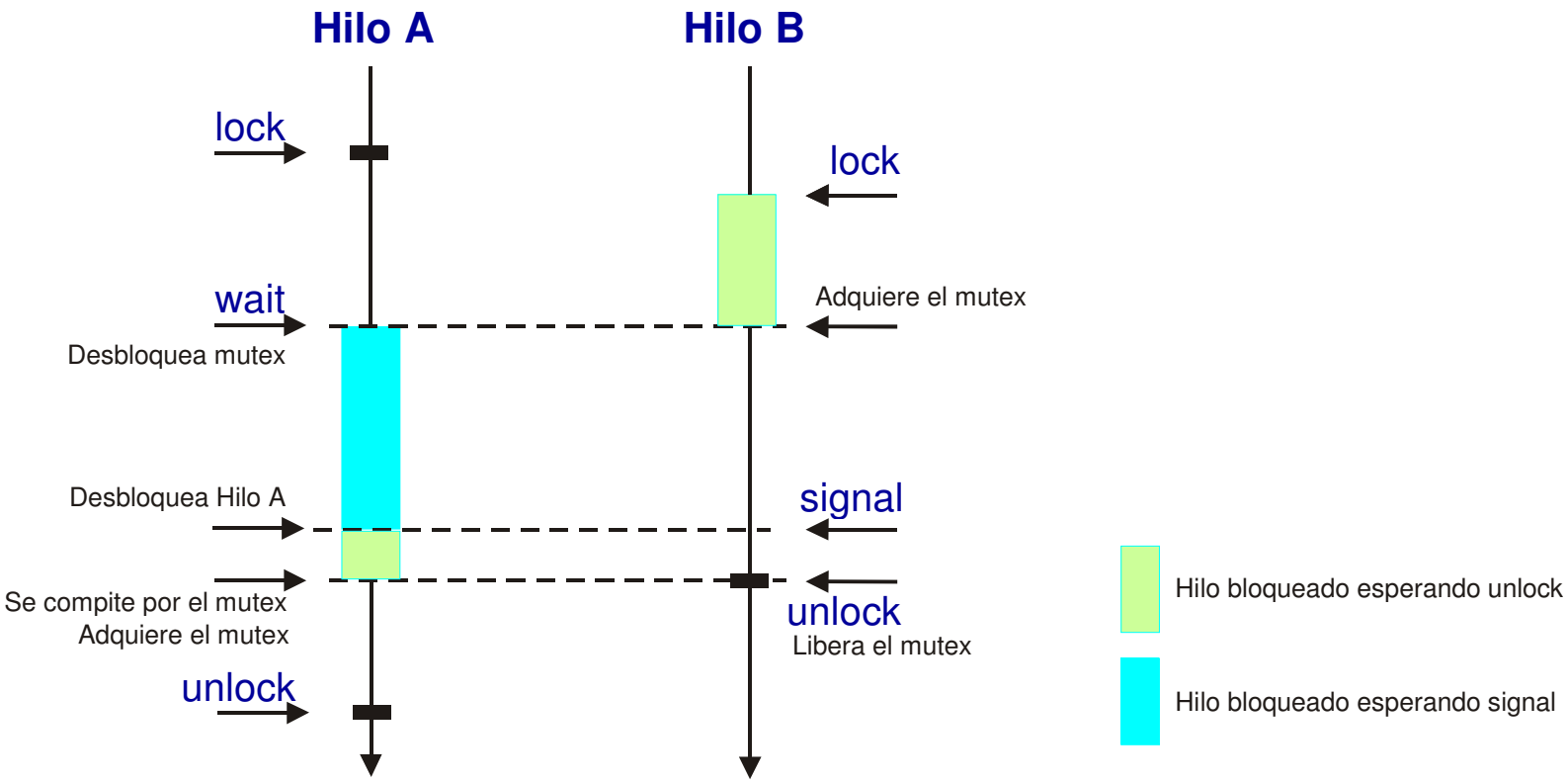
```
lock(mutex);
...
while (condicion == FALSE)
        wait(condition, mutex);
condicion = FALSE;
...
unlock(mutex);
```

!!Importante!!

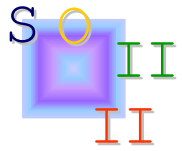
Hilo B

```
lock(mutex);
...
condicion = TRUE;
signal(condition, mutex);
unlock(mutex);
```

Uso de mutex y variables condicionales



Servicios POSIX sobre mutex



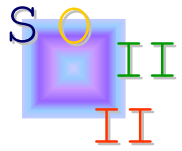
- **Identificación de un mutex en POSIX:**
  - Variable del tipo `pthread_mutex_t`
- **Funciones sobre mutex en POSIX:**

```
int pthread_mutex_init   (pthread_mutex_t * mutex,
                          pthread_mutexattr_t * attr);
int pthread_mutex_destroy(pthread_mutex_t *mutex);
int pthread_mutex_lock   (pthread_mutex_t *mutex);
int pthread_mutex_unlock (pthread_mutex_t *mutex);
```

devuelven:

- Si todo ha ido bien: 0
- Si error: -1

Inicialización y destrucción de mutex



- **Inicialización de un mutex:**
  - Sintaxis:

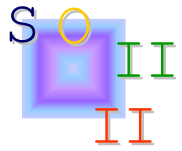
```
int pthread_mutex_init(pthread_mutex_t *mutex,
                       pthread_mutexattr_t * attr);
```

  - Descripción:
    - Inicializa un mutex identificado a través de `mutex` con los atributos especificados a través de `attr` (atributos por defecto si NULL)
- **Destrucción de un mutex:**
  - Sintaxis:

```
int pthread_mutex_destroy(pthread_mutex_t *mutex);
```

  - Descripción:
    - Destruye un mutex identificado a través de `mutex`

Operaciones *lock* y *unlock*



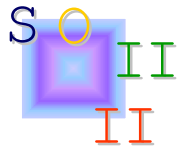
- **Operación `lock` sobre un mutex POSIX:**
  - Sintaxis:

```
int pthread_mutex_lock(pthread_mutex_t *mutex);
```

- **Operación `unlock` sobre un mutex POSIX :**
  - Sintaxis:

```
int pthread_mutex_unlock(pthread_mutex_t *mutex);
```

Servicios POSIX sobre variables de condición



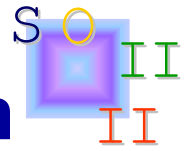
- **Identificación de una variable de condición en POSIX:**
  - Variable del tipo `pthread cond t`
- **Funciones sobre variables de condición en POSIX :**

```
int pthread_cond_init      (pthread_cond_t * cond,
                            pthread_condattr_t * attr);
int pthread_cond_destroy   (pthread_cond_t *cond);
int pthread_cond_wait      (pthread_cond_t * cond,
                            pthread_mutex_t * mutex);
int pthread_cond_signal    (pthread_cond_t * cond);
int pthread_cond_broadcast (pthread_cond_t * cond);
```

devuelven:

- Si todo ha ido bien: 0
- Si error: -1

Inicialización y destrucción de vars. de condición



- **Inicialización de una variable de condición:**
  - Sintaxis:

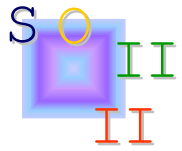
```
int pthread_cond_init(pthread_cond_t *cond,
                      pthread_condattr_t *attr);
```

  - Descripción:
    - Inicializa una variable de condición identificada a través de `cond` con los atributos especificados a través de `attr` (atributos por defecto si NULL)
- **Destrucción de una variable de condición:**
  - Sintaxis:

```
int pthread_cond_destroy(pthread_cond_t *cond);
```

  - Descripción:
    - Destruye una variable de condición identificada a través de `cond`

Operación *wait* sobre variables de condición

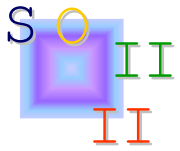


- **Operación `wait` sobre una variable de condición:**
  - Sintaxis:

```
int pthread_cond_wait(pthread_cond_t*cond,
                      pthread_mutex_t*mutex);
```

  - Descripción:
    - Suspende al hilo hasta que otro hilo señaliza la variable condicional `cond`
    - Se libera el `mutex` atómicamente
    - Cuando se despierta el hilo vuelve a competir por el `mutex`

Operación *signal* sobre variables de condición



- **Operación `signal` sobre una variable de condición:**
  - Sintaxis:

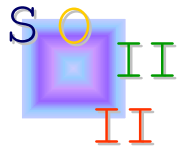
```
int pthread_cond_signal(pthread_cond_t * cond);
```

  - Descripción:
    - Se reactiva uno de los hilos que están suspendidos en la variable condicional `cond`
    - No tiene efecto si no hay ningún hilo esperando (diferente a los semáforos)
- **Operación `broadcast` sobre una variable de condición:**
  - Sintaxis:

```
int pthread_cond_broadcast(pthread_cond_t * cond);
```

  - Descripción:
    - Todos los hilos suspendidos en la variable condicional `cond` se reactivan
    - No tiene efecto si no hay ningún hilo esperando

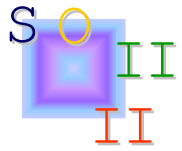
Servicios POSIX sobre mutex



- **Ejemplo 1: El problema del productor-consumidor con buffer limitado (circular):**

```
#define MAX_BUFFER          1024        /* Tamaño del buffer */
#define DATOS_A_PRODUCIR    100000      /* Datos a producir */
pthread_mutex_t mutex;                  /* Mutex que controla acceso al buffer */
pthread_cond_t no_lleno;                /* Controla el llenado del buffer */
pthread_cond_t no_vacio;                /* Controla el vaciado del buffer */
int n_elementos;                        /* Número de elementos en el buffer */
int buffer[MAX_BUFFER];                 /* Buffer común */
```

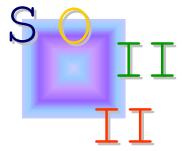
Servicios POSIX sobre mutex



- **Ejemplo 1 (cont.):**

```
void Productor(void)  {                                  /* Código del Productor */
   int dato, i ,pos = 0;
   for(i=0; i < DATOS A PRODUCIR; i++ )  {
        dato = i;                                        /* Producir dato */
        pthread_mutex_lock(&mutex);                      /* Acceder al buffer */
        while (n elementos == MAX BUFFER)                /* Si buffer lleno */
            pthread_cond_wait(&no_lleno, &mutex);        /* se bloquea */
        buffer[pos] = i;
        pos = (pos + 1) % MAX BUFFER;
        n elementos ++;
        pthread_cond_signal(&no_vacio);                  /* Buffer no vacío */
        pthread_mutex_unlock(&mutex);
    }
    pthread exit(0);
}
```

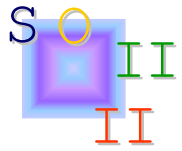
Servicios POSIX sobre mutex



- **Ejemplo 1 (cont.):**

```
void Consumidor(void)   {                                   /* Código del Consumidor */
int dato, i ,pos = 0;
    for(i=0; i < DATOS A PRODUCIR; i++ ) {
        pthread_mutex_lock(&mutex);                         /* Acceder al buffer */
        while (n elementos == 0)                            /* Si buffer vacío */
            pthread_cond_wait(&no_vacio, &mutex);           /* se bloquea */
        dato = buffer[pos];
        pos = (pos + 1) % MAX BUFFER;
        n elementos --;
        pthread_cond_signal(&no_lleno);                     /* Buffer no lleno */
        pthread_mutex_unlock(&mutex);
        printf("Consume %d \n", dato);                      /* Consume dato */
    }
    pthread exit(0);
}
```

Servicios POSIX sobre mutex



- **Ejemplo 1 (cont.):**

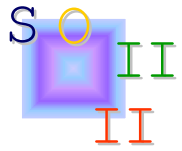
```
main(){
    pthread_t th1, th2;
    pthread_mutex_init(&mutex, NULL);
    pthread_cond_init(&no_lleno, NULL);
    pthread_cond_init(&no_vacio, NULL);

    pthread_create(&th1, NULL, Productor, NULL);
    pthread_create(&th2, NULL, Consumidor, NULL);

    pthread_join(th1, NULL);
    pthread_join(th2, NULL);
    pthread_mutex_destroy(&mutex);
    pthread_cond_destroy(&no_lleno);
    pthread_cond_destroy(&no_vacio);

    exit(0);
}
```

Tema 3. Concurrencia entre procesos

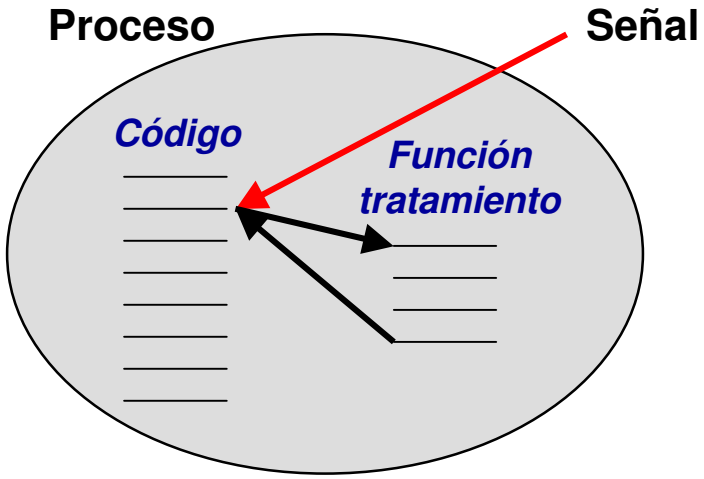


Índice

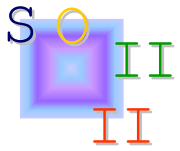
- Procesamiento concurrente
- El problema de la sección crítica
- Semáforos
- Mutex y variables de condición
- Señales
- Paso de mensajes
- Monitores
- Mecanismos de concurrencia en sistemas Unix
- Mecanismos de concurrencia en Windows NT/2000
- Mecanismos de concurrencia en Minix

Señales

- **¿Qué es una señal?**
  - Herramienta de comunicación/sincronización entre procesos
  - Interrupción a un proceso provocada por ese proceso, por otro proceso o por el SO

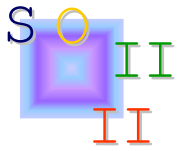


Señales



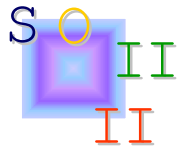
- Permiten informar a un proceso que ha ocurrido un evento:
  - Evento de error:
    - Generado por el proceso en ejecución
    - Violación de segmento, instrucción ilegal, escritura en zona de sólo lectura, etc.
  - Evento asíncrono:
    - Generado externamente al proceso pero relacionado con él
    - Muerte de un hijo, alarma de un reloj, desconexión de un terminal, etc.

Señales



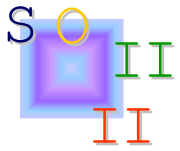
- **¿Quién envía y a quién se envía?**
  - Proceso → Proceso
    - Proceso → Proceso con el mismo identificador de usuario
      → Grupo de procesos correspondiente
    - Superusuario → Cualquier proceso
  - SO → Proceso
    - Excepción de programa convertida en señal al proceso que ha causado la excepción
- **Señales y estado de procesos**
  - Una señal puede ser enviada en cualquier momento a un proceso en cualquier estado
  - Si el proceso no está en ejecución la señal ha de guardarse hasta que reanude su ejecución

Señales



- **Recepción de una señal:**
  - Activación del bit asociado a la señal en la componente correspondiente de la estructura del proceso (que recibe)
- **Reconocimiento y tratamiento de una señal:**
  - El reconocimiento y tratamiento de una señal no se realiza necesariamente de manera inmediata tras su llegada
  - El SO chequea la recepción de una señal (y la trata si procede), por ejemplo:
    - Cuando selecciona un nuevo proceso a ejecutar
    - Al retornar de una llamada al sistema
  - Las señales no tienen prioridades
  - Cada proceso tiene información de cómo tratar a cada señal

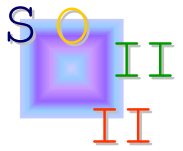
Señales



- **Reacción de un proceso ante la recepción de una señal:**
  - Ignorar la señal (desecharla)
  - Bloquear la señal (hasta que sea desenmascarada)
  - Ejecutar la acción por defecto asociada a la señal:
    - Finalizar proceso (matarlo) → Generación de fichero “core”
      → Uso de `kill`
    - Ignorar la señal
    - Parar proceso
    - Reanudar proceso
  - Ejecutar la rutina de tratamiento de la señal (*función del manejador* o, simplemente, *manejador de señal*)

Bloquear señal ≠ Ignorar señal

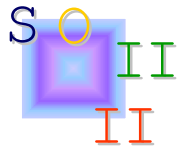
Señales



- **Implementación de señales:**
  - El SO tiene que:
    - Recordar las señales enviadas y bloqueadas por cada proceso
    - Chequear las señales recibidas
    - Determinar si la señal es aceptada:
      - Si la señal no está bloqueada por el proceso destino Y
      - Si la señal no es ignorada por el proceso destino

      Y en ese caso tratarla:
      - Ejecutar la acción por defecto asociada O
      - Ejecutar el correspondiente manejador de señal

Recordando ...



- **Implementación de señales en Linux:**
  - Campos del descriptor de proceso relacionados con señales:

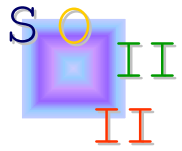
estructura `task_struct`

```
struct sigset_t signal;      /* Mapa de bits de señales recibidas  */
struct sigset_t blocked;     /* Mapa de bits de señales bloqueadas */
struct sigset_t sigpending;  /* Mapa de bits de señales no bloqueadas
                                y pendientes */
struct signal_struct *sig;   /* Manejadores de señales */
```

```
struct signal_struct {
    atomic_t count;
    struct sigaction action[NSIG]; };

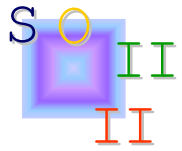
struct sigaction {
    void (*sa_handler)();    /* Manejador de señal */
    sigset_t sa_mask;        /* Señales bloqueadas durante ejecución
                                del manejador */
    int sa_flags;            /* Operaciones especiales */
    };
```

Señales definidas en POSIX



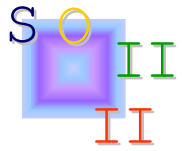
Señal	Descripción	
SIGTERM	Terminación	
SIGHUP	Desconexión del teminal de control	
SIGQUIT	Terminación interactiva	**Ctrl+|**
SIGINT	Atención interactiva	**Ctrl+C**
SIGALRM	Fin de temporización	
SIGKILL	Terminación	
SIGSTOP	Parada	
SIGTSTP	Parada interactiva	**Ctrl+S**
SIGCONT	Continuación interactiva	**Ctrl+Q**

Señales definidas en POSIX



Señal	Descripción
SIGCHLD	Indica terminación de un hijo
SIGILL	Instrucción de HW ilegal
SIGFPE	Operación aritmética errónea (p.e., divsión por cero)
SIGSEGV	Referencia a memoria inválida
SIGBUS	Error de bus
SIGPIPE	Error en tubería sin lectores
SIGUSR1	Definida por usuario
SIGUSR2	Definida por usuario

Señales definidas en POSIX



- **Listado de las posibles señales del sistema:**

```
$ kill -l
```

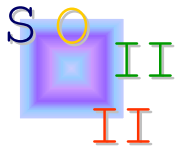
- **Listado de de los caracteres especiales que generan señales:**

```
$ sttv -a
```

- Algunas señales (como SIGKILL y SIGSTOP) no pueden ser ignoradas ni armadas.

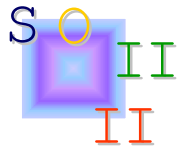
Armar: Asignar un manejador

Señales y alarmas en POSIX



- **Aspectos relacionados con una señal:**
  - ¿Cómo enviar una señal?
  - ¿Cómo armar una señal?
  - ¿Cómo esperar señales?
- **Aspectos relacionados con una alarma:**
  - ¿Cómo activar una alarma?

Servicios POSIX sobre señales



- **Envío de señales:**
  - Sintaxis:

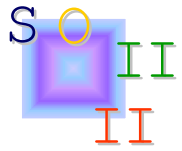
```
int kill (pid_t pid, int sig);
```

  - Descripción:
    - Envía la señal número `sig` al proceso o grupo de procesos especificado por `pid`

Si `pid` >0	al proceso con identificativo `pid`
Si `pid` =0	a todos los procesos del mismo grupo de procesos que el del proceso que envía
Si `pid` =-1 y UID=root	a todos los procesos
Si `pid` =-1 y UID≠root	a todos los procesos con UID=EUID del proceso que envía

  - Devuelve:
    - Si todo ha ido bien: 0
    - Si error: -1

Servicios POSIX sobre señales



- **Envío de señales (cont.):**
  - Ejemplo:

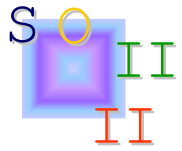
```
kill (getppid(), SIGTERM);
```

  - El comando del intérprete de órdenes `kill` invoca a la función `kill`:

```
$ kill -9 1023
```

¿Qué hace?

Servicios POSIX sobre señales



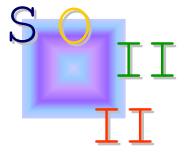
- **Armado de señales:**
  - Sintaxis:

```
void (*signal(int signum, void (*manejador)(int)))(int);
```

  - Descripción:
    - Asocia a la señal número `signum` la acción a realizar ante la recepción de la señal especificada en el segundo parámetro, que puede ser:
      - `SIG_DFL`: Acción por defecto
      - `SIG_IGN`: Ignorar señal
      - Una función a ejecutar especificada por el usuario
  - Devuelve:
    - Si todo ha ido bien: El anterior manejador de señal
    - Si error: -1 (SIGERR)

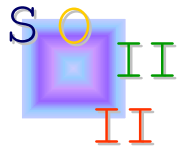
¿Para qué?

Servicios POSIX sobre señales



- **Armado de señales (cont.):**
  - Tras la invocación del manejador:
    - Si se sigue la semántica BSD no se reasigna el manejador de señal por defecto
    - Si se sigue la semántica SysV, sí

Servicios POSIX sobre señales



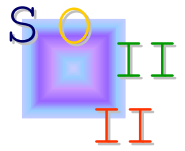
- **Espera de señales:**
  - Sintaxis:

```
int pause (void);
```

  - Descripción:
    - Bloquea al proceso que la invoca hasta que llegue cualquier señal no ignorada
  - Devuelve:
    - Siempre –1 (no tiene ningún significado)

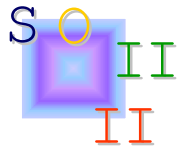
pause vs. sigsuspend

Otros servicios POSIX sobre señales



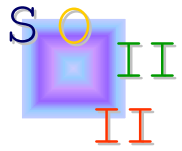
- **Conjuntos de señales:**
  - Un proceso puede realizar operaciones sobre un conjunto de señales (de tipo **`sigset_t`**)
- **Función `sigaction`:**
  - Arma una señal
- **Función `sigprocmask`:**
  - Modifica la máscara de señales (bloqueadas) del proceso que la invoca
- **Función `sigsuspend`:**
  - Bloquea al proceso que la invoca hasta que llegue una señal especificada
- **Función `sleep`:**
  - Despierta al proceso que la invoca cuando ha transcurrido el tiempo establecido como argumento o cuando se recibe una señal

Servicios POSIX sobre alarmas



- **Activación de una alarma:**
  - Sintaxis:
    ```
    unsigned int alarm (unsigned int seconds);
    ```
  - Descripción:
    - Envía al proceso que la invoca la señal `SIGALRM` tras `seconds` segundos
    - Si `seconds=0` cancela cualquier petición anterior
  - Las peticiones hechas con `alarm` no se apilan

Servicios POSIX sobre señales



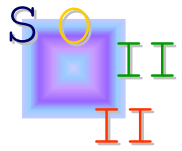
- **Ejemplo 1:**

```
#include <signal.h>
main()
{ int pid;

  if ((pid=fork()) == 0)
  { while(1)
      { printf("HIJO: PID = %d\n",getpid());
        sleep(1);
      }
  }
  sleep(5);
  printf("PADRE: Terminación del proceso hijo %d\n",pid);
  kill(pid,SIGTERM);
  exit(0);
}
```

¿Ejecuta el proceso hijo la instrucción `sleep(5)` ?

Servicios POSIX sobre señales



- **Ejemplo 2:**

```
#include <stdio.h>
#include <stdlib.h>
#include <signal.h>

void gestor_alarma()
{ printf("Activada\n"); }

main()
{
   signal(SIGALRM,gestor_alarma);
   for (;;)   {
     alarm(2);
     pause();
   }
}
```

¿Cómo podemos finalizar la ejecución del proceso?

Servicios POSIX sobre señales

- **Ejemplo 3:**

  Si el SO NO restaura el manejador por defecto al invocar la rutina de tratamiento

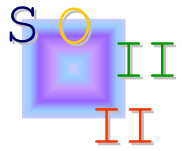
```
#include <signal.h>

void sigint_handler (int sig)
{ printf ("Recibida la señal número %d\n",sig); }

main ()
{
  if (signal(SIGINT,sigint_handler) == SIG_ERR)
  {  perror("signal");
     exit(-1);
  }
  while (1)
  {
     printf("Espero a que pulses Ctrl-C\n");
     sleep(999);
  }
}
```

¡Ojo!

Servicios POSIX sobre señales



- **Ejemplo 3 (cont.):**

  Si el SO restaura el manejador por defecto al invocar la rutina de tratamiento

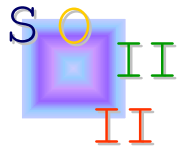
```
#include <signal.h>

void sigint_handler (int sig)
{
  printf ("Recibida la señal número %d\n",sig);
  if (signal(SIGINT,sigint_handler) == SIG_ERR)
  {  perror("signal"); exit(-1);}
}

main ()
{
  if (signal(SIGINT,sigint_handler) == SIG_ERR)
  {  perror("signal");
     exit(-1);
  }
  while (1)
  {  printf("Espero a que pulses Ctrl-C\n");
     sleep(999);
  }
}
```

¿Qué ocurre si se pulsa Ctrl-C?

Servicios POSIX sobre señales



- **Ejemplo 3 (cont.):**

  Si el SO restaura el manejador por defecto al invocar la rutina de tratamiento:

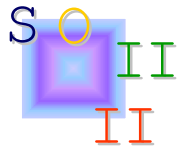
  Para deshabilitar la recepción de una señal del mismo tipo durante el tratamiento:

```
void sigint_handler (int sig)
{
  if (signal(SIGINT,SIG_IGN) == SIG_ERR)
  { perror("signal"); exit(-1); }

  printf ("Recibida la señal número %d\n",sig);

  if (signal(SIGINT,sigint_handler) == SIG_ERR)
  { perror("signal"); exit(-1); }
}
```

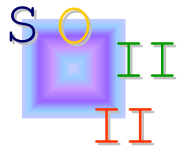
Tema 3. Concurrencia entre procesos



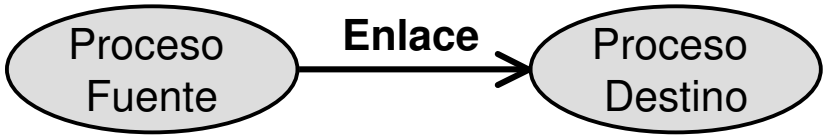
Índice

- Procesamiento concurrente
- El problema de la sección crítica
- Semáforos
- Mutex y variables de condición
- Señales
- Paso de mensajes
- Monitores
- Mecanismos de concurrencia en sistemas Unix
- Mecanismos de concurrencia en Windows NT/2000
- Mecanismos de concurrencia en Minix

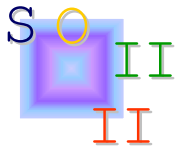
Paso de mensajes



- La comunicación la realiza el SO
- Realizado mediante la utilización (ejecución) de las primitivas **send** y **receive**
- Entre el emisor y el receptor debe haber un **enlace de comunicaciones**

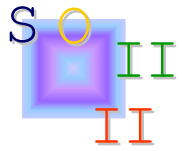


Paso de mensajes



- **Características del paso de mensajes:**
  - Nominación del proceso emisor/receptor
    - Nominación directa
    - Nominación indirecta
  - Capacidad del enlace
    - Nula
    - Finita
    - Infinita
  - Primitivas de envío y recepción bloqueantes o no bloqueantes

Paso de mensajes



- **Algunas combinaciones de las anteriores características:**
  - **Nominación directa, capacidad de enlace nula y sentencias bloqueantes (“Rendezvous”, citas o reencuentros):**
    - Si se hace un SEND antes de llegar un RECEIVE el proceso que hizo el SEND se bloquea hasta que llegue un RECEIVE
    - Si se hace un RECEIVE antes de llegar un SEND el proceso que hizo el RECEIVE se bloquea hasta que llegue un SEND

Paso de mensajes

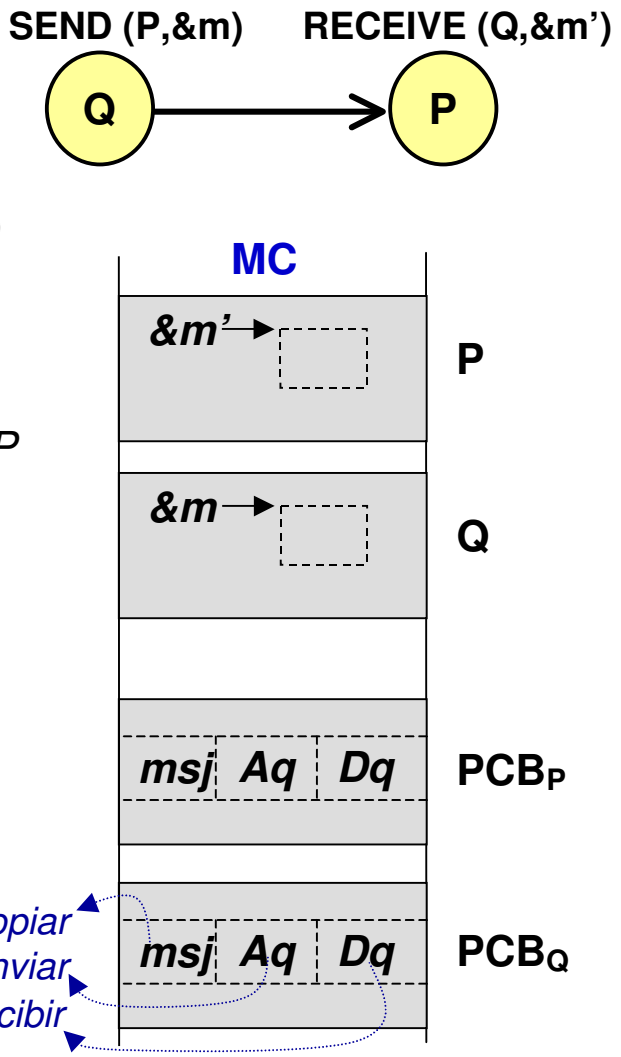
Rendezvous

Q: SEND(P,&m)

SI *P* está esperando el mensaje de *Q* (*PCB_P.Dq=Q*)
ENTONCES Enviar (copiar) mensaje de *&m* a *PCB_P.msj (=&m')*
Desbloquear *P*
SINO *PCB_Q.msj=&m*
PCB_Q.Aq=P
Bloquear *Q* hasta que llegue *RECEIVE(Q,&m')* de *P*

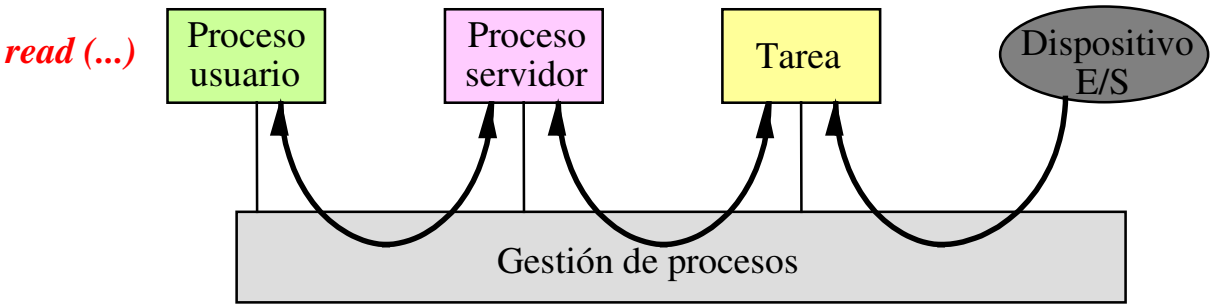
P: RECEIVE(Q,&m')

SI *Q* ha intentando enviar un mensaje a *P* (*PCB_Q.Aq=P*)
ENTONCES Enviar (copiar) mensaje de *PCB_Q.msj (=&m)* a *&m'*
Desbloquear *Q*
SINO *PCB_P.msj=&m'*
PCB_P.Dq=Q
Bloquear *P* hasta que llegue *SEND(P,&m)* de *Q*

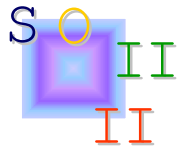


Recordando ...

- **Núcleo de Minix:**
  - Capas 1 y 2 del modelo multinivel
  - **Los subsistemas cliente y servidor se comunican mediante paso de mensajes mediante la técnica *rendezvous***
    - Minix reconvierte las llamadas al sistema en mensajes cuyo destino es el gestor de memoria o el gestor de ficheros
    - Toda interrupción HW es reconvertida en un mensaje
  - El mecanismo de mensajes se lleva a cabo en el núcleo



Paso de mensajes en Minix

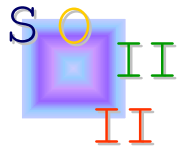


- **Sincronización y comunicación entre procesos de Minix:**
  - Paso de mensajes siguiendo la técnica “rendezvous”
- **Primitivas de envío y recepción:**
  - *send (dest,&m)* Envía el mensaje *m* a *dest*
  - *receive (fte,&m)* Recibe el mensaje *m* de *fte*
  - *send_rec (fte_dest,&m)* Envía el mensaje *m* a *fte_dest* y espera recibir contestación del mismo proceso

Puede ser ANY

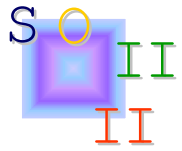
- Un proceso (o tarea) puede enviar o recibir mensajes de otro proceso (o tarea) del mismo nivel, de su nivel inmediatamente anterior o de su nivel inmediatamente posterior

Paso de mensajes en Minix



- **Primitiva** *send_rec (fte_dest,&m)*:
  - Equivale a hacer:
    - *send (fte_dest,&m)*
    - *receive (fte_dest,&m)*
  - Reescribe el mensaje en *&m*

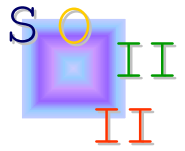
Recordando ...



- **Descriptor de un proceso de Minix:**

```
struct proc{
   ...
   struct proc *p_callera;   /* head of list of procs wishing to send */
   struct proc *p_sendlink;  /* link to next proc wishing to send */
   message *p_messbuf;       /* pointer to message buffer */
   int p_getfrom;            /* from whom does process want to receive? */
   ...
} proc[NR_TASKS+NR_PROCS];
```

Paso de mensajes en Minix



- **Implementación de un mensaje:**

```
typedef struct
{int m_source;
 int m_type;
 union { mess_1 m_m1; mess_2 m_m2; mess_3 m_m3;
         mess_4 m_m4; mess_5 m_m5; mess_6 m_m6; } m_u;
} message;
```

Cabecera fija

Parte variable

i ≡ *entero*
p ≡ *puntero*
l ≡ *long*
ca ≡ *palabras*
F ≡ *función*

m_m1
m_source
m_type
m1_i1
m1_i2
m1_i3
m1_i4
m1_p1
m1_p2
m1_p3

m_m2
m_source
m_type
m2_i1
m2_i2
m2_i3
m2_l1
m2_l2
m2_p1

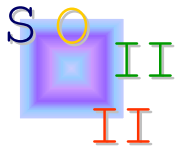
m_m3
m_source
m_type
m3_i1
m3_i2
m3_p1
m3_ca1

m_m4
m_source
m_type
m4_l1
m4_l2
m4_l3
m4_l4

m_m5	
m_source	
m_type	
m5_c2	m5_c1
m5_i1	
m5_i2	
m1_l1	
m5_l2	
m5_l3	

m_m6
m_source
m_type
m6_i1
m6_i2
m6_i3
m6_l1
m6_F1

Paso de mensajes en Minix



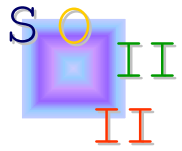
- **Implementación de la llamada al sistema `write`:**

PROCESO USUARIO

```
main (argc, argv, envp)
{ ...
  write (n,s,sizeof(s));      { build mess (&m,WRITE,n,
                                            s,sizeof(s));
                                send rec (FS,&m);

  ...
                              { build mess (&m,EXIT,0);
  exit (0);                   send rec (MM,&m);
}
```

Paso de mensajes en Minix

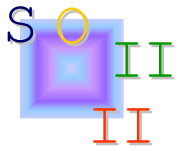


- **Implementación de la llamada al sistema `write` (cont.):**

PROCESO SERVIDOR (MM o FS)

```
main ()
{ init_mmfs ();
  while (TRUE) {
        receive (ANY,&mess);
        caller=mess.m_source;
        switch (mess.m_type)
        {       ...
                case READ:   do_write (&mess); break;
                case WRITE:  do_write (&mess); break;
                ...
                case EXIT:   do_exit (&mess); break;
        }
        build_reply (&mess);
        send (caller,&mess);
  }
do_write ()
{ ...
  send_rec (FLOPPY,&mess);
  ...
} }
```

Paso de mensajes en Minix

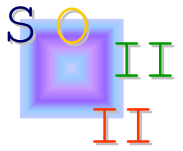


- **Implementación de la llamada al sistema `write` (cont.):**

TAREA FLOPPY

```
floppy_task ()
{ init floppy ();
  while (TRUE) {
        receive (ANY,mess);
        caller=mess.m_source;
        switch (mess.m_type)
        {       ...
                case DISK_READ:  do_rdrw (&mess); break;
                ...
                case DISK_WRITE: do_rdrw (&mess); break;
        }
        build_reply (&mess);
        send (caller,&mess);
  }
do_rdwr ()
{ /* Programar controlador de discos */
  receive (HW,&mess); /* Esperar fin de operación */
  /* Tratamiento de errores */
}
}
```

Paso de mensajes en Minix



- **Implementación de la llamada al sistema `write (cont.)`:**

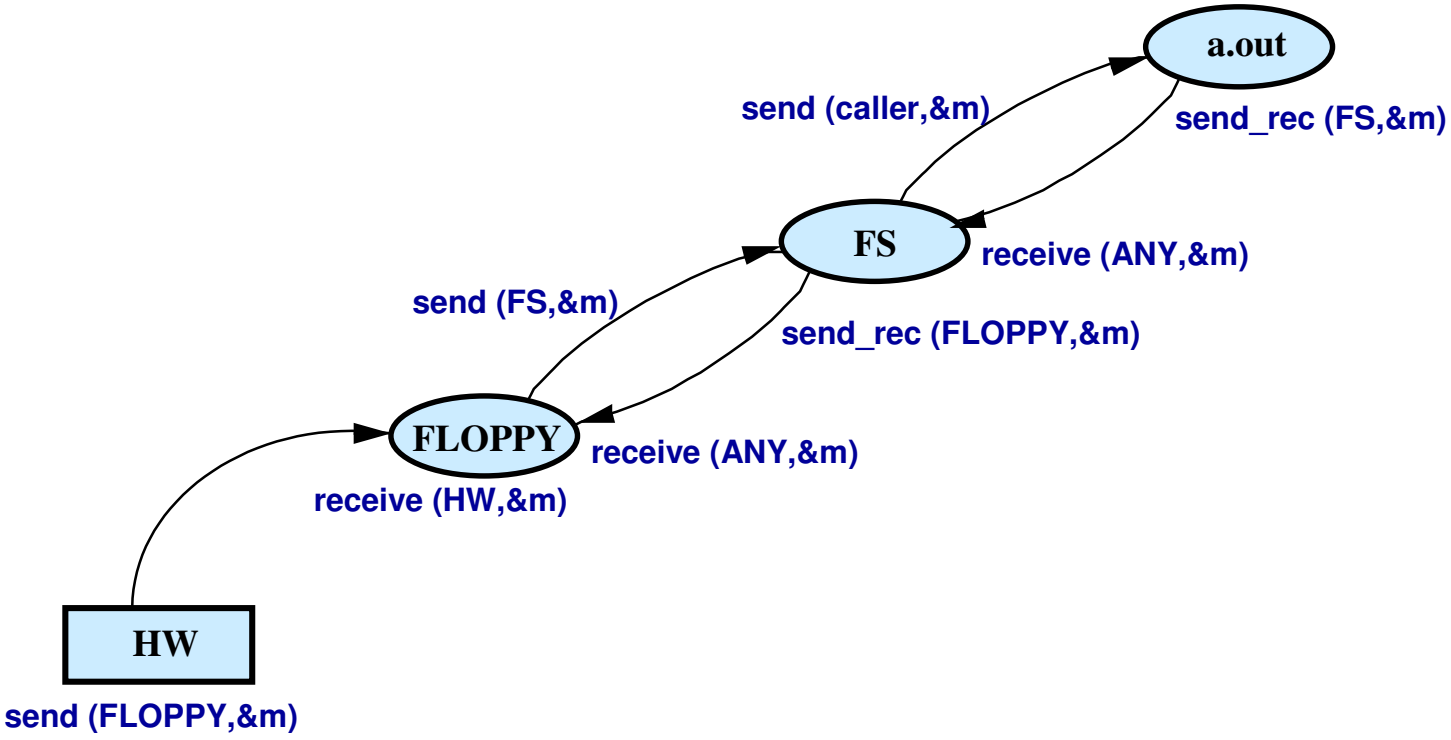
PROCESO HW

```
hw ()
{ /* Espera interrupción */
  switch (int)
  {     case DISK INT:   send (FLOPPY,&m); break;
        case TERM INT:   send (TERM,&m);   break;
        case CLOCK INT:  send (CLOCK,&m);  break;
  }
}
```

- HW no es un proceso real
- Toda interrupción HW es reconvertida en un mensaje
- Las rutinas de servicio de interrupción se encargan del envío de estos mensajes

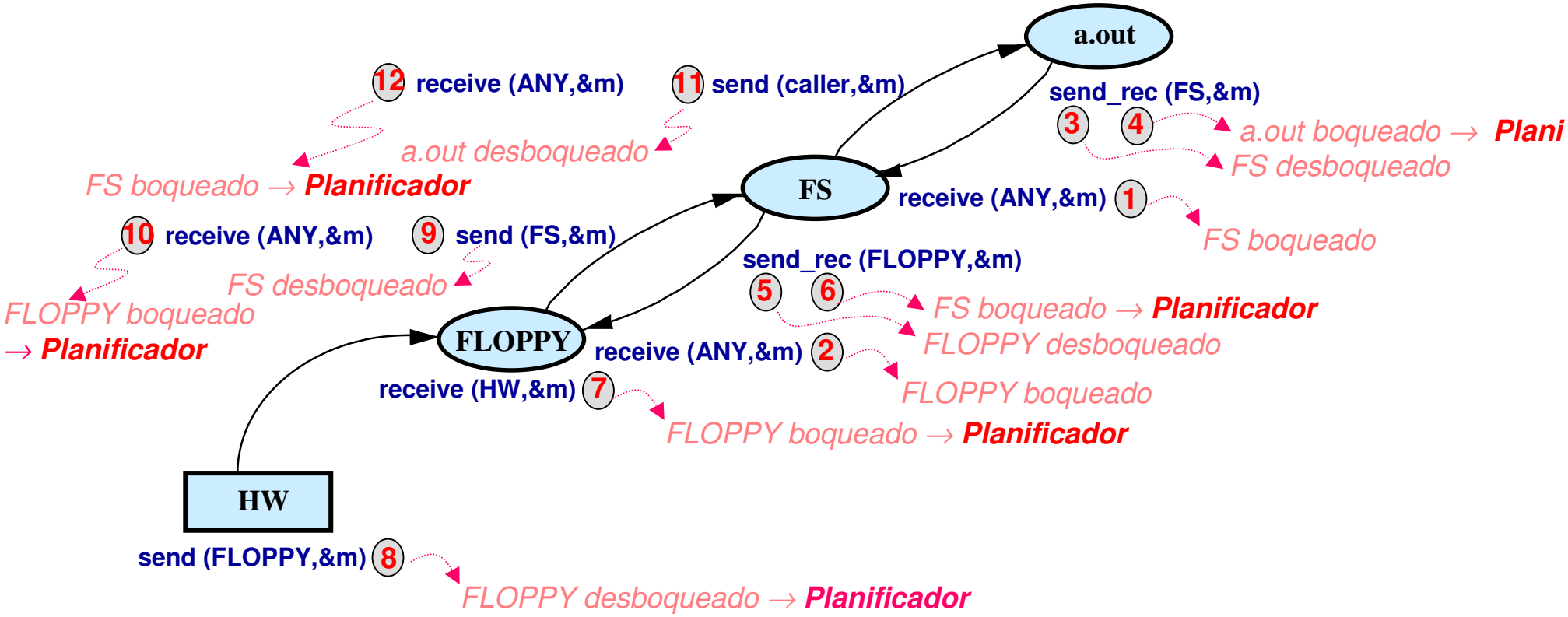
Paso de mensajes en Minix

- **Implementación de la llamada al sistema `write (cont.)`:**

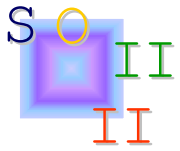


Paso de mensajes en Minix

- **Implementación de la llamada al sistema `write (cont.)`:**



Recordando ...

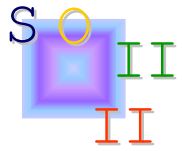


- **El planificador de Minix:**

  Se invoca al planificador (rutina *pick_proc*) cuando:

  - Se bloquea un proceso (en ejecución) con SEND o RECEIVE
  - Tras cada interrupción
  - Tras la finalización del cuantum de CPU de un proceso de usuario
  - Al finalizar un proceso

Recordando ...



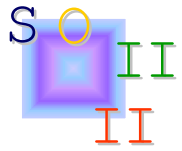
- **Implementación del planificador de Minix:**

```
PUBLIC pick_proc ()

{   register int q;/* Cola a usar:                                    */
                   /*   - Cola de tareas:              TASK Q     */
                   /*   - Cola de procesos servidor:   SERVER Q   */
                   /*   - Cola de procesos de usuario: USER Q     */

    if (rdy_head [TASK Q] != NIL_PROC)                q = TASK Q;
         else if (rdy_head [SERVER Q]  != NIL_PROC)   q = SERVER Q;
                   else                               q = USER Q;
   prev_proc = cur_proc;
   if (rdy_head [q]  != NIL_PROC)
   {    cur_proc = rdy_head [q]; /* Someone is runable */
   } else{
        cur_proc = IDLE;          /* No one id runable */
   }
}
```

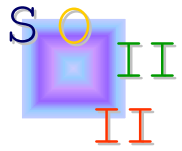
Tema 3. Concurrencia entre procesos



Índice

- Procesamiento concurrente
- El problema de la sección crítica
- Semáforos
- Mutex y variables de condición
- Señales
- Paso de mensajes
- Monitores
- Mecanismos de concurrencia en sistemas Unix
- Mecanismos de concurrencia en Windows NT/2000
- Mecanismos de concurrencia en Minix

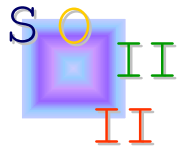
Semáforos vs. Monitores



- **Inconvenientes de los semáforos:**
  - No soportan abstracciones de datos
  - Precaución en la colocación de semáforos:
    - **Bloqueos mutuos**: Dos procesos o más esperan indefinidamente la ocurrencia de un suceso que sólo uno de los procesos que esperan puede causar
    - **Inanición**: Los procesos esperan indefinidamente dentro del semáforo a que otro proceso haga una operación **señal**

$$\textit{Bloqueo mutuo} \Rightarrow \textit{Inanición}$$
$$\overset{?}{\Leftarrow}$$

Semáforos vs. Monitores



- **Inconvenientes de los semáforos (cont.):**
  - Ejemplo:
    - Sean **A** y **B** dos procesos cada uno de los cuales acceden a dos semáforos **S** y **Q** inicializados a 1:

Proceso A	**Proceso B**
wait (S);	wait (Q);
wait (Q);	wait (S);
...	...
signal (S);	signal (Q);
signal (Q);	signal (S);

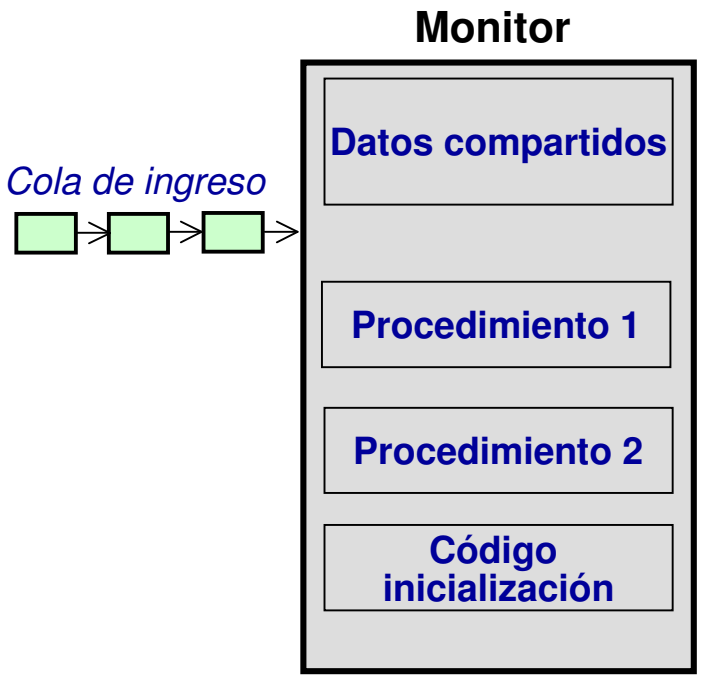
    - Secuencia de ejecución:

```
A:  wait (S);
B:  wait (Q);
A:  wait (Q);   El proceso A espera a que B ejecute signal (Q);
B:  wait (S);   El proceso B espera a que A ejecute signal (S);
```

Bloqueo mutuo e inanición

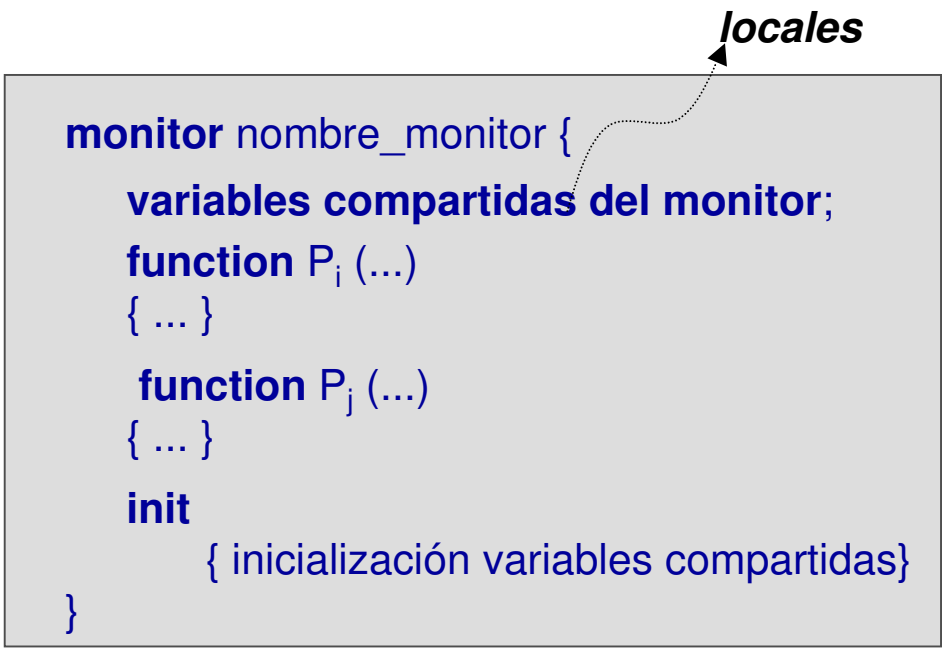
Monitores

- **¿Qué es un monitor?**
  - Construcción de sincronización de un lenguaje de alto nivel que permite la compartición (segura) de un tipo abstracto de datos entre procesos concurrentes
  - Paquete o módulo especial que contiene estructuras de datos, variables y procedimientos/funciones
  - El compilador (o el SO) realiza la exclusión mutua y el programador no necesita implementarla explícitamente

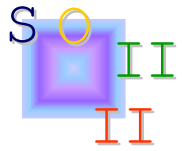


Monitores

- **Especificación de un monitor:**



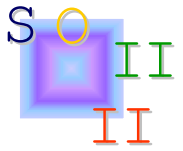
Monitores



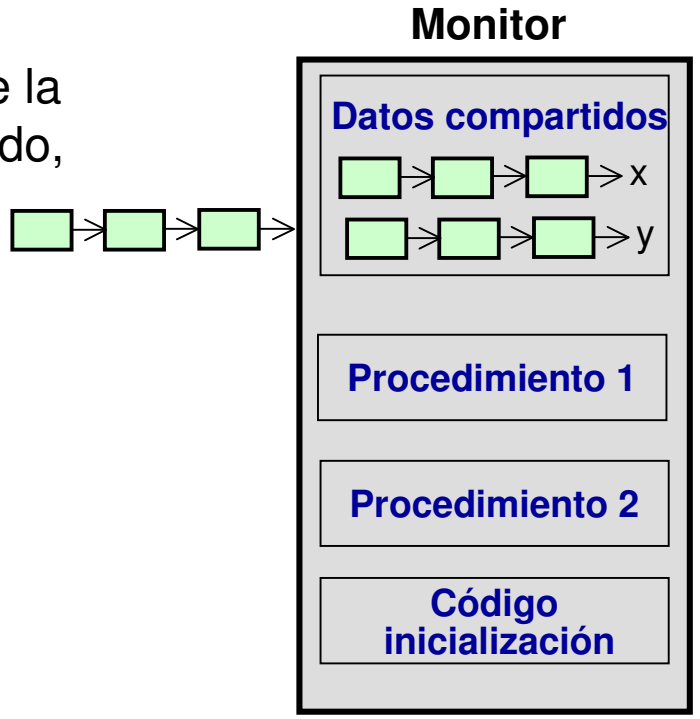
- **¿Qué proporcionan los monitores?**
  - **Encapsulamiento de datos**:
    - Sólo se puede acceder al objeto monitor mediante las operaciones definidas en él
  - **Exclusión mutua**:
    - La construcción del monitor garantiza que sólo podrá estar activo un proceso a la vez dentro del monitor
    - Si el monitor está libre, el proceso entra; si no, el proceso se suspende hasta que el proceso que está usando el monitor salga
  - **Sincronización**:
    - Para conseguir que un proceso espere dentro del monitor, se define un tipo especial de variable **condition**:

```
condition x;
```

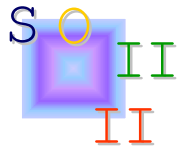
Monitores



- **Operaciones definidas sobre variables tipo condition:**
  - **delay (x)**: El proceso que invoca esta operación se suspende ante la condición **x** hasta que otro proceso invoque resume (x)
  - **resume (x)**: Activa un proceso bloqueado ante la condición **x**. Si no hay ningún proceso bloqueado, esta operación no tiene efecto
    - Otros nombres de estas operaciones:
      - ✓ delay(x) ≡ stop(x) ≡ wait(x)
      - ✓ resume(x) ≡ restart(x) ≡ signal(x)



Monitores



- **Operaciones definidas sobre variables tipo condition (cont.):**

  Problema:

  - Al desbloquear un proceso bloqueado por la condición **x**, habrá dos procesos activos dentro del monitor → No se cumple la exclusión mutua dentro del monitor

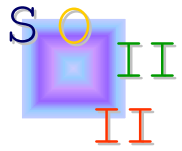
  Solución:

  - Utilización de cola de procesos bloqueados por **resume (x)**

    Hacer que la última instrucción que se ejecuta en una rutina del monitor sea **resume (x)**

    Cuando se libera el monitor pasar a la cola de procesos listos un proceso de la cola de procesos bloqueados por **resume (x)**

Monitores

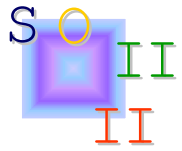


- **Ejemplo: El problema del productor_consumidor con buffer limitado mediante monitores:**

```
monitor Productor_Consumidor {
    condition lleno, vacio;                    /* lleno≡ Buffer lleno; vacio≡ Buffer vacío */
    int nelementos;                            /* nelementos ≡ Num. elementos en buffer */
    /* Definición del buffer */

    void Dejar () {
        if (nelementos == N) delay(lleno);     /* Espera porque el buffer está lleno */
        /* Dejar elemento en el buffer */
        nelementos = nelementos + 1;
        if (nelementos == 1) resume(vacio);    /* Avisa que el buffer ya no está vacío */
    }
```

Monitores



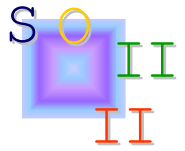
- **El problema del productor_consumidor con buffer limitado mediante monitores (cont.):**

```
 void Coger () {
      if (nelementos == 0) delay(vacio);          /* Espera porque el buffer está vacio */
      /* Coger elemento del buffer */
      nelementos = nelementos - 1;
      if (nelementos == N-1) resume(lleno);       /* Avisa que el buffer ya no está lleno */
}

init {
       nelementos=0;

}     /* Fin del monitor */
```

Monitores



- **El problema del productor_consumidor con buffer limitado mediante monitores (cont.):**

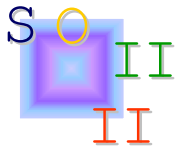
```
void Productor () {

    while (1) {
        /* Producir elemento */
        Productor_Consumidor.Dejar();
        }
}

void Consumidor()  {

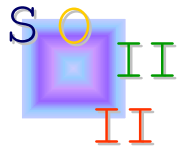
    while (1) {
        Productor_Consumidor.Coger();
        /* Consumir elemento */
        }
}
```

Monitores



- **Inconvenientes de los monitores:**
  - Necesario un lenguaje de alto nivel que soporte monitores:
    Pascal Concurrente, Modula-2 y 3, Java
  - Funcionan en multiprocesadores con memoria común pero no con multiprocesadores con memoria local

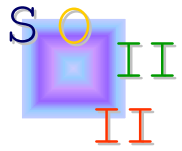
Tema 3. Concurrencia entre procesos



Índice

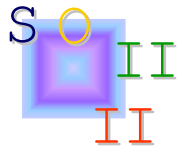
- Procesamiento concurrente
- El problema de la sección crítica
- Semáforos
- Mutex y variables de condición
- Señales
- Paso de mensajes
- Monitores
- Mecanismos de concurrencia en sistemas Unix
- Mecanismos de concurrencia en Windows NT/2000
- Mecanismos de concurrencia en Minix

Concurrencia en sistemas Unix



- **Mecanismos de concurrencia:**
  - Tuberías
  - Memoria compartida
  - Mensajes
  - Semáforos
  - Señales
  - Mutex y variables de condición (para hilos)

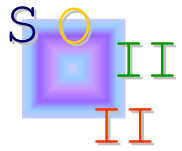
Tema 3. Concurrencia entre procesos



Índice

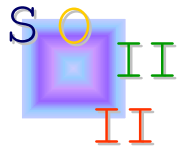
- Procesamiento concurrente
- El problema de la sección crítica
- Semáforos
- Mutex y variables de condición
- Señales
- Paso de mensajes
- Monitores
- Mecanismos de concurrencia en sistemas Unix
- Mecanismos de concurrencia en Windows NT/2000
- Mecanismos de concurrencia en Minix

Sincronización en Windows NT/2K



- Un hilo ha de esperar en ocasiones a que ocurra un determinado evento (finalización de una operación de E/S, finalización de un hilo, etc.) antes de continuar su ejecución → Sincronización con el evento
- La sincronización se lleva a cabo mediante *objetos de sincronización* específicos como son:
  - Objetos de tipo proceso, hilo, fichero, evento, semáforo, alarma y mutante (mutex)

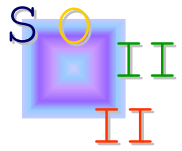
Recordando ...



- **Tipos de objetos en Windows NT/2K:**

Objeto	Descripción
Proceso	Invocación de un programa con su espacio de direccionamiento y recursos requeridos
Hilo	Entidad ejecutable de un proceso
Sección	Región de memoria compartida
Fichero	Instancia de un fichero abierto o de un dispositivo de E/S
Puerto	Destino de los mensajes transferidos entre procesos
Acceso	Información de seguridad sobre un usuario conectado
Evento	Mecanismo para indicar que ha ocurrido un evento del sistema
Mutante	Mecanismo para conseguir exclusiones mutua
Alarma	Contador alarma que registra el paso del tiempo

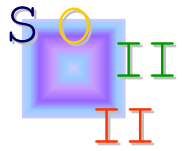
Sincronización en Windows NT/2K



- Un objeto de sincronización puede estar en dos posibles situaciones:
  - Marcado ("signaled")
  - No marcado ("nonsignaled"
- Un hilo puede estar bloqueado por un objeto en estado no marcado.

  Cuando el objeto pasa a situación de marcado se liberan inmediatamente uno o todos los hilos que esperan a (bloqueados por) dicho objeto

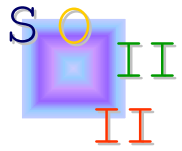
Sincronización en Windows NT/2K



- Objetos de sincronización:

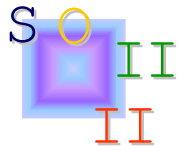
Tipo de objeto	Pasa a estado marcado cuando ...	Efecto sobre hilos que esperan
Proceso	Finalizan todos sus hilos	Todos se liberan
Hilo	Finaliza el hilo	Todos se liberan
Semáforo	El contador del semáforo llega a 0	Todos se liberan
Mutante	El hilo propietario u otro hilo libera mutante	Se libera un hilo
Evento	Ocurre el evento	Todos se liberan
Alarma	Llega el tiempo de activación o expira	Todos se liberan

Comunicación en Windows NT/2K



- **Tipos de comunicación:**
  - Comunicación entre hilos de un proceso:
    - Mediante espacio de direccionamiento (compartido) del proceso
  - Comunicación entre procesos (servidor y cliente):
    - Mediante paso de mensajes

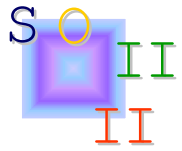
Tema 3. Concurrencia entre procesos



Índice

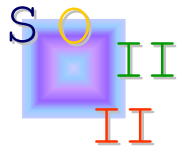
- Procesamiento concurrente
- El problema de la sección crítica
- Semáforos
- Mutex y variables de condición
- Señales
- Paso de mensajes
- Monitores
- Mecanismos de concurrencia en sistemas Unix
- Mecanismos de concurrencia en Windows NT/2000
- Mecanismos de concurrencia en Minix

Concurrencia en Minix



- **Mecanismos de concurrencia:**
  - Tuberías
  - Memoria compartida
  - Mensajes
  - Señales

Concurrencia: Ejercicios



- **Ejercicio 1:**

  ¿Cuál de las siguientes afirmaciones sobre los mecanismos de sincronización de procesos es cierta?

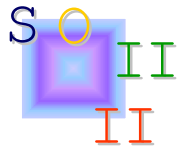
  (a) Cualquier mecanismo es válido sobre cualquier plataforma.

  (b) El paso de mensajes no se puede utilizar para comunicar procesos que se ejecutan en un computador con una sola CPU.

  (c) La espera activa es el mecanismo más ineficiente en el uso de la CPU.

  (d) Con semáforos nunca se puede dar un interbloqueo.

Concurrencia: Ejercicios



- **Ejercicio 2:**

  ¿Cuál de las siguientes afirmaciones relacionadas con tuberías (pipes) es falsa?

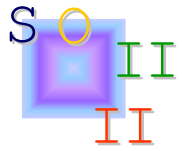
  (a) Si la tubería está vacía, el lector se queda bloqueado hasta que algún escritor.

  (b) Las operaciones de lectura pueden tener tamaños distintos a las operaciones de escritura.

  (c) En POSIX dos procesos que quieren comunicarse ejecutan ambos la llamada *pipe*.

  (d) Un escritor puede escribir en la tubería aunque un lector no haya ejecutado una lectura del mismo.

Concurrencia: Ejercicios



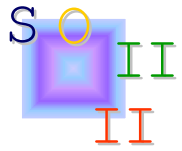
- **Ejercicio 3:**

  Dos procesos de Linux, A y B, comparten una tubería p, siendo A el proceso que lee de dicha tubería y B el que escribe en ella. En un momento determinado la tubería contiene 78 bytes y el proceso A ejecuta las dos funciones siguiente:

```
read (p[0], &c1, 36);
read (p[0], &c2, 85);
```

  ¿Qué ocurre al ejecutarse cada una de estas funciones? Asumir que no se produce un cambio de contexto durante dichas ejecuciones.

Concurrencia: Ejercicios



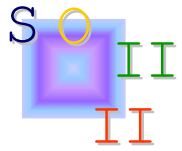
- **Ejercicio 4:**

  Estudiar la validez del siguiente código:

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int main(int argc, char *argv[])
{ int n, i, fd[2], pid, estado;
  n=atoi(argv[1]);
  for (i=1; i<n; i++)
  { pipe(fd);
    if (fork()==0) {
        close(fd[1]);
        close(0);
        dup(fd[0]);
        close(fd[0]);
    } else {
        close(fd[0]);
        close(1);
        dup(fd[1]);
        close(fd[1]);
        if (i==1){
           pid=getpid();
           write(1,&pid,sizeof(pid));
           wait(&estado);
        } else {
           read (0,&pid,sizeof(pid));
           write(1,&pid,sizeof(pid));
           wait(&estado);
        }
        exit(0);
    }
  }
  read (0,&pid,sizeof(pid));
  printf("%d\n",pid);
  exit(0);
}
```

Concurrencia: Ejercicios

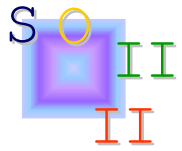


- **Ejercicio 5:**

  ¿Cuál es el número máximo de procesos que pueden ejecutar sin bloquearse una operación `wait` sobre un semáforo que se inicializó con un valor de 4?

  ¿Y cuál es el número máximo de procesos que pueden bloquearse en dicho semáforo?

Concurrencia: Ejercicios

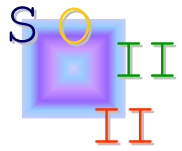


- **Ejercicio 6:**

  Realizar un programa que cree dos hilos: El primero de ellos generará 100 números pares y el segundo otros 100 números impares, mostrándose todos éstos por pantalla con la restricción de que no pueden aparecen dos números pares seguidos o dos números impares seguidos.

  Utilizar semáforos como herramienta de sincronización.

Concurrencia: Ejercicios



- **Ejercicio 6 (sol.):**

```
#include <pthread.h>
#include <semaphore.h>
#include <stdio.h>

sem_t ImprPar, ImprImpar;

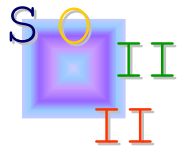
void *GeneraPares(void *arg)
{
  int i;

  for (i=0; i<200; i=i+2)
  {
    sem_wait(&ImprPar);
    printf("(Hilo %d):%d\n",
           pthread_self(),i);
    sem_post(&ImprImpar);
  }
  pthread_exit(0);
}
```

```
void *GeneraImpares(void *arg)
{
  int i;

  for (i=1; i<30; i=i+2)
  {
    sem_wait(&ImprImpar);
    printf("(Hilo %d):%d\n",
           pthread_self(),i);
    sem_post(&ImprPar);
  }
  pthread_exit(0);
}
```

Concurrencia: Ejercicios



- **Ejercicio 6 (sol.):**

```
main()
{
  pthread_t Pares, Impares;

  sem_init(&ImprPar,0,1);
  sem_init(&ImprImpar,0,0);

  pthread_create(&Pares, NULL, GeneraPares, NULL);
  pthread_create(&Impares, NULL, GeneraImpares, NULL);

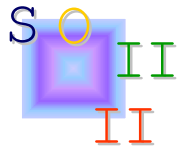
  pthread_join(Pares,NULL);
  pthread_join(Impares,NULL);

  sem_destroy(&ImprPar);
  sem_destroy(&ImprImpar);

  exit(0);

}
```

Concurrencia: Ejercicios



- **Ejercicio 6 (sol.):**

  ¿Qué habría ocurrido si inicializamos a 1 ambos semáforos?

  Sea la siguiente notación de instrucciones:

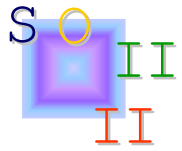
```
    void *GeneraPares(void *arg)
    {
       int i;

P.0    for (i=0; i<200; i=i+2)
       {
P.1      sem_wait(&ImprPar);
P.2      printf("(Hilo %d):%d\n",
                 pthread_self(),i);
P.3      sem_post(&ImprImpar);
       }
       pthread_exit(0);
    }
```

```
    void *GeneraImpares(void *arg)
    {
      int i;

I.0    for (i=1; i<30; i=i+2)
       {
I.1      sem_wait(&ImprImpar);
I.2      printf("(Hilo %d):%d\n",
                 pthread_self(),i);
I.3      sem_post(&ImprPar);
       }
       pthread_exit(0);
    }
```

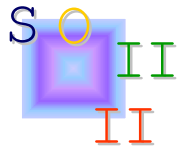
Concurrencia: Ejercicios



- **Ejercicio 6 (sol.):**

Instrucción	ImprPar.v	.L	ImprImpar.v	.L	i(Par)	i(Impar)	printf i
	1		1				
P.0					0		
P.1	0						
I.0						1	
I.1			0				
I.2							**1**
I.3	1						
P.2							**0**
P.3			1				
P.0					2		
P.1	0						
P.2							**2**
P.3			2				

Concurrencia: Ejercicios



- **Ejercicio 7:**

  Se desea que el inicio de una determinada actividad de un proceso, al que denominaremos P1, comience después de que finalice una actividad de otro proceso denotado por P2. Indicar el código necesario para asegurar este comportamiento utilizando como herramienta de sincronización:

  (a)Tuberías.

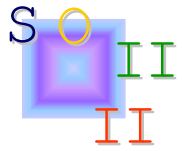
  (b) Semáforos.

  (c) Mutex y variables de condición.

  (d) Señales POSIX.

  (d) Mensajes con la técnica “rendezvous”.

Concurrencia: Ejercicios



- **Ejercicio 8:**

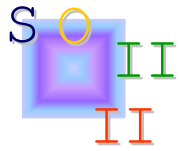
  Sea el siguiente código que bloquea a un hilo hasta que se cumpla una determinada condición:

```
1: pthread_mutex_lock(&mutex);
2: while (ocupado == true)
3:    pthread_cond_wait(&cond, &mutex);
4: ocupado = true;
5: pthread_mutex_unlock(&mutex);
```

  Y sea el siguiente código que permite desbloquear al hilo que ejecute el código anterior:

```
6: pthread_mutex_lock(&mutex);
7: ocupado = false;
8: pthread_cond_signal(&cond);
9: pthread_mutex_unlock(&mutex);
```

Concurrencia: Ejercicios



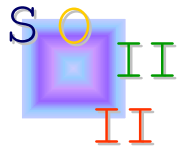
- **Ejercicio 8 (cont.):**

  Supongamos que el valor de la variable `ocupado` es `true` y que existen dos hilos en el sistema (denotados por A y B).

  (a) Asumiendo que el hilo A ejecuta el primer fragmento de código y el hilo B el segundo, mostrar algunas secuencias de ejecución posibles.

  (b) ¿Es posible la secuencia de ejecución A1 A2 B6 B7 B8 A2 A3 B9, donde Xi denota que el hilo X ejecuta la instrucción i?

Concurrencia: Ejercicios

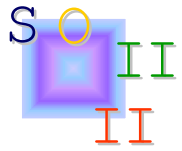


- **Ejercicio 8 (sol.):**

  (a) A1 A2 A3 B6 B7 B8 B9 A2 A4 A5
  A1 B6 A2 A3 B7 B8 B9 A2 A4 A5
  B6 B7 B8 B9 A1 A2 A4 A5
  B6 B7 A1 B8 B9 A2 A4 A5

  (b) La secuencia A1 A2 B6 B7 B8 A2 A3 B9 no es posible.
  Cuando el hilo B6 ejecuta la instrucción 6, el mutex está bloqueado.

Concurrencia: Ejercicios

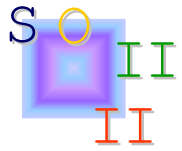


- **Ejercicio 9:**

  Realizar un programa en C que cree un proceso hijo que ejecute el comando que se le pasa como argumento al programa con los parámetros especificados en la invocación de éste a partir del segundo argumento.

  El proceso padre esperará a que el hijo finalice con la restricción de que, por requisitos del sistema, no puede estar esperando más de 10 segundos. Una vez transcurrido ese tiempo, deberá abortar la ejecución del proceso hijo.

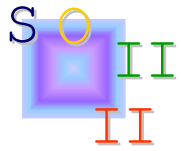
Concurrencia: Ejercicios



- **Ejercicio 9 (sol.):**

```
#include <signal.h>
int pid;
void gestor_alarma()
{  kill(pid,SIGKILL); }
main(int argc, char * argv[])
{ int estado;
  if ((pid=fork()) != 0)
  {     signal(SIGALRM,gestor_alarma);
        alarm (10);
        wait(&estado);
        alarm (0);
        exit(0);
  } else { /* HIJO */
        execvp(argv[1],&argv[1]);
        perror("Error en el exec");
        exit(-1);
  }
}
```

Concurrencia: Ejercicios



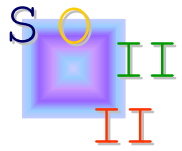
- **Ejercicio 10:**

  ¿Cuál es el funcionamiento del siguiente programa asumiendo que la función `signal` sigue la semática

  (a) BSD.

  (b) SysV.

Concurrencia: Ejercicios



- **Ejercicio 10 (cont.):**

```
#include <signal.h>

int cont=0;

void manejador_sigUSR1 (int sig)

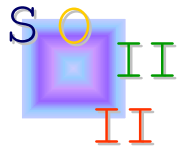
{ printf ("Recibida la señal número %d\n",sig);
  cont++;
}

void manejador_sigUSR2 (int sig)

{ printf ("Recibida la señal número %d\n",sig);
  printf ("Se han recibido %d señales SIGUSR1\n",cont);
  exit(0);
}

main ()
{
  if (signal(SIGUSR1,manejador_sigUSR1) == SIG_ERR)
  {  perror("signal USR1");      exit(-1); }
  if (signal(SIGUSR2,manejador_sigUSR2) == SIG_ERR)
  {  perror("signal USR2");      exit(-1); }
  while (1);
}
```

Concurrencia: Ejercicios



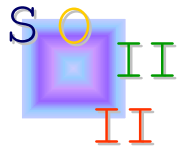
- **Ejercicio 11:**

  Realizar un programa en C que cree un proceso hijo que ejecute el comando que se le pasa como argumento al programa con los parámetros especificados en la invocación de éste a partir del segundo argumento.

  El proceso padre esperará a que el hijo finalice y, hasta que esto ocurra, no deberá ser interrumpido por las señales SIGINT y SIGQUIT generables desde teclado.

  El proceso hijo deberá mantener el tratamiento de señales original del padre.

Concurrencia: Ejercicios



- **Ejercicio 11 (sol.):**

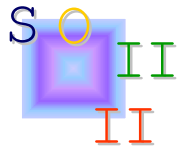
```
#include <signal.h>
main(int argc, char * argv[])

{ int pid, estado;
  void (*del)(), (*quit)();

  del =signal(SIGINT, SIG_IGN);
  quit=signal(SIGQUIT,SIG_IGN);

  if ((pid=fork()) != 0)
  {     wait(&estado);
        printf("Mi hijo ha finalizado\n");
        signal(SIGINT,del);
        signal(SIGQUIT,quit);
        exit(0);
  } else {
        signal(SIGINT,del);
        signal(SIGQUIT,quit);
        execvp(argv[1],&argv[1]);
        perror("Error en el exec");
        exit(-1);
  }
}
```

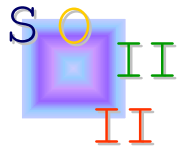
Concurrencia: Ejercicios



- **Ejercicio 12:**

  Si se modifica el programa del ejercicio anterior como se especifica a continuación, ¿qué ocurre si se pulsa dos veces Ctrl-C? ¿Y si se pulsa tres?

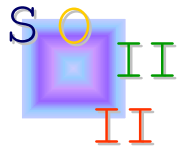
Concurrencia: Ejercicios



- **Ejercicio 12 (cont.):**

```
#include <signal.h>
main(int argc, char * argv[])
{ int pid, estado;
  void (*del)(), (*quit)();
  del =signal(SIGINT, SIG_IGN);
  quit=signal(SIGQUIT,SIG_IGN);
  if ((pid=fork()) != 0)
  {     wait(&estado);
        printf("Mi hijo ha finalizado\n");
        pause();
        exit(0);
  } else {
        signal(SIGINT,del);
        signal(SIGQUIT,quit);
        execvp(argv[1],&argv[1]);
        perror("Error en el exec");
        exit(-1);
  }
}
```

Concurrencia: Ejercicios



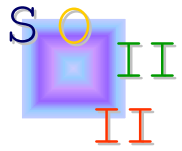
- **Ejercicio 13:**

  Escribir un programa en C que active unos manejadores de las señales SIGINT, SIGQUIT y SIGILL. Las acciones a realizar por dichos manejadores serán:

  (a) Para SIGINT y SIGQUIT, abortar el proceso devolviendo un estado de error.

  (b) Para SIGILL, imprimir un mensaje de instrucción ilegal y terminar.

Concurrencia: Ejercicios

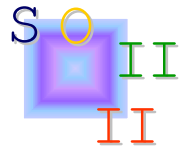


- **Ejercicio 14:**

  Escribir un programa en C que lea un fichero de números enteros, cuyo nombre se pasa como primer argumento al programa, y busque si se encuentra en él el número que se le pasa como segundo argumento al programa.

  El proceso debe leer el fichero sin verse afectado por las señales SIGINT y SIGQUIT. Y activará una alarma de 20 segundos que, si se dispara, debe imprimir un mensaje de error y terminar.

Concurrencia: Ejercicios



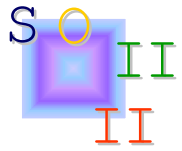
- **Ejercicio 15:**

  Explicar paso a paso cómo se ejecutaría una llamada al sistema `read (fd, vector, 524)` que realiza una operación de lectura sobre un disco de un SO **minix**.

- **Ejercicio 16:**

  Explicar qué posibles situaciones podrían ocurrir cuando el proceso *Memory Manager* (MM) de **minix** ejecuta una sentencia `send rec (FLOPPY, &m)` (hasta la finalización de ésta).

Concurrencia: Ejercicios



- **Ejercicio 17:**

  Si *P* y *Q* son dos procesos de usuario de **minix** que se encuentran en la cola correspondiente de procesos listo, indicar qué ocurrirá desde que el primero de ellos entra en ejecución hasta que acaba la ejecución de ambos.

Proceso P

```
/* Construye mensaje m */
send-rec (Q,&m);
printf ("A");
```

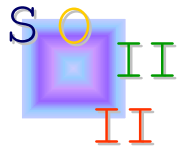
Proceso Q

```
receive (P,&mm);
printf ("B");
send_rec (R,&mm);
printf ("C");
send (P,&mm);
```

Proceso R

```
receive (Q,&mm);
printf ("D");
send (Q,&mm);
```

Concurrencia: Ejercicios



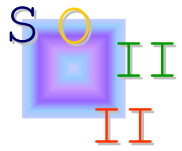
- **Ejercicio 17 (solución):**

  (i) Supongamos que **CPL = PQR**:

CPL(antes)	Proceso	Instrucción	CPL(después)	CPB
POR	P	send-rec(O) → P bloqueado	OR	P
OR	O	rec (P) → P desbloqueado	ORP	λ
		print (B)		
		send-rec(R) → O bloqueado	RP	O
RP	R	rec (O) → O desbloqueado	RPO	λ
		print (D)		
		send (O) → R bloqueado	PO	R
PO	P	send-rec(O) → P bloqueado	O	RP
O	O	send-rec(R) → R desbloqueado	OR	P
		print (C)		
		send (P) → P desbloqueado	ORP	λ
		exit → Fin O	RP	
RP	R	exit → Fin R	P	λ
P	P	print (A)	P	λ
	P	exit → Fin P	λ	

Por tanto, se imprime: BCDA

Concurrencia: Ejercicios

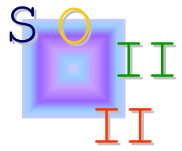


- **Ejercicio 17 (solución):**

  (ii) Si **CPL** $\neq$ **PQR** también se imprime BCDA

¿Qué ocurre si se agota el quantum del proceso en ejecución?

Concurrencia: Ejercicios



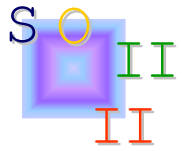
- **Ejercicio 18:**

  Indicar cuántas veces, en qué situaciones y por qué se invoca al planificador de **minix** al intentar un proceso de usuario escribir un entero en un fichero. Suponer que todas las colas de procesos listos están vacías en ese instante.

- **Ejercicio 19:**

  ¿Por qué se invoca al planificador de **minix** después de la atención de una interrupción? Indíquese algún ejemplo que ilustre la respuesta.

Concurrencia : Ejercicios



- **Ejercicio 20:**

  ¿Cuándo una operación **`delay`** sobre una variable de sincronización de un **monitor** bloquea al proceso que la ejecuta?

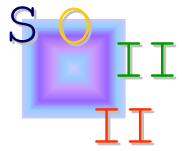
  (a) Siempre

  (b) Siempre que hay un proceso esperando ejecutar una rutina del monitor

  (c) Siempre que la variable tenga valor negativo

  (d) Siempre que no se haya hecho una operación **`resume`** sobre la misma variable

Concurrencia : Ejercicios



- **Ejercicio 21:**

  ¿Cuántos procesos pueden desbloquearse simultáneamente debido a la realización de una operación **`resume`** sobre una variable de condición de un **monitor**?

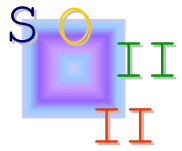
  (a) Todos los que comparten el monitor menos uno

  (b) Todos los que están bloqueados en dicha variable

  (c) Dos

  (d) Uno

Concurrencia : Ejercicios



- **Ejercicio 22:**

  ¿Cuántos procesos pueden desbloquearse simultáneamente debido a la realización de una operación **`signal`** sobre un **semáforo**?

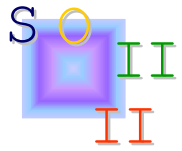
  (a) Todos los que están bloqueados en dicho semáforo

  (b) Todos los que comparten el monitor menos uno

  (c) Dos

  (d) Uno

Concurrencia : Ejercicios



- **Ejercicio 23:**

  Suponiendo que en una variable de condición de un **monitor** hay N procesos bloqueados, ¿cuántas operaciones **`resume`** sobre dicha variable puede ejecutar un proceso antes de abandonar el monitor para activar a los procesos que en ella hay bloqueados?

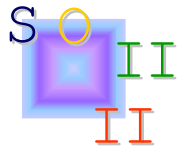
  (a) 1

  (b) N

  (c) N - 1

  (d) 2N

Concurrencia : Ejercicios



- **Ejercicio 24:**

  ¿Cuántos procesos (en cualquier estado) pueden tener como máximo la copia del contador del programa del descriptor de proceso apuntando a una instrucción que pertenezca a un **monitor**?

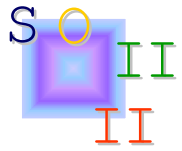
  (a) Tantos como compartan el monitor menos uno

  (b) Tantos como compartan el monitor

  (c) Dos

  (d) Sólo uno

Concurrencia : Ejercicios

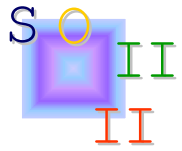


- **Ejercicio 25:**

  Suponer que hay dos tipos de procesos TipoA y TipoB que acceden a datos comunes a todos ellos de tal forma que, en un instante determinado únicamente pueden acceder a los datos procesos del mismo tipo.

  Escríbase el código correspondiente para resolver el problema de acceso a los datos utilizando **monitores** como herramienta de sincronización.

Concurrencia : Ejercicios



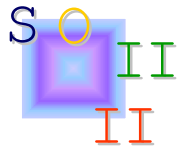
- **Ejercicio 25 (solución):**

```
monitor Ejercicio {
   condition procsA;          /* procsA ≡  Hay procesos tipoA accediendo a datos */
   condition procsB;          /* procsB ≡  Hay procesos tipoB accediendo a datos */
   int nA:                    /* nA ≡  Procesos tipoA accediendo a datos */
   int nB:                    /* nB ≡  Procesos tipoB accediendo a datos */

void AccedeA () {
   if (nB>0) {
      delay (procsB);          /* Espera porque hay procesos tipoB accediendo */
      nA=nA+1:
      release (procsB);        /* Avisa que no hay procesos tipoB accediendo */
   } else nA:=nA+1:
}

void SaleA () {
   nA=nA-1;
   if (nA=0) release (procsA); /* Avisa que no hay procesos tipoA accediendo */
}
```

Concurrencia : Ejercicios



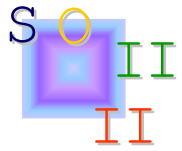
- **Ejercicio 25 (solución):**

```
void AccedeB () {
   if (nA>0) {
      delay (procsA);                  /* Espera porque hay procesos tipoA accediendo */
      nB=nB+1;
      release (procsA);                /* Avisa que no hay procesos tipoA accediendo */
   } else nB=nB+1;
}

void SaleB() {
   nB=nB-1;
   if (nB==0) release (procsB);        /* Avisa que no hay procesos tipoB accediendo */
}

init {
   nA=0;  nB:=0;
}  /*  Fin del monitor */
```

Concurrencia : Ejercicios



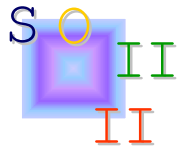
- **Ejercicio 25 (solución):**

```
void ProcesoA (int i);
{  Ejercicio.AccedeA;
   /* Acceder a Datos */
   Ejercicio.SaleA;
}


void ProcesoB (int i);
{  Ejercicio.AccedeB;
   /* Acceder a Datos */
   Ejercicio.SaleB;
}


parbegin
{  for (i=0; i<MaxA; i++) ProcesoA (i);
   for (i=0; i<MaxB; i++) ProcesoB (i);
}
```

Concurrencia : Ejercicios



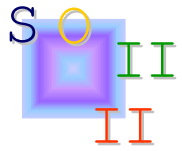
- **Ejercicio 26:**

  Una carretera atraviesa un túnel cuya anchura no permite que se crucen dos vehículos que circulan en sentido contrario. El control del tráfico se realiza mediante unas barreras en ambas entradas al túnel (entrada norte y entrada sur) guiadas por un sistema informático. Para controlar dichas barreras se proporciona al programador la llamada `LevantaBarrera (identificador)`, que levanta una de las barreras de entrada al túnel (`BarreraNorte` o `BarreraSur`), asegurándose además que pasa un único vehículo (en sentido de entrada) cada vez que se ejecuta dicha función.

  También se han colocado sensores en la entrada y salida del túnel que detectan cuándo llega un nuevo vehículo a uno de los extremos y cuándo sale uno que acaba de atravesar el túnel. Cuando tienen lugar dichas situaciones, los sensores generan uno de los siguientes procesos:

  - `LlegaVehiculoANorteTúnel`
  - `LlegaVehiculoASurTúnel`
  - `SaleVehiculoANorteTúnel`
  - `SaleVehiculoASurTúnel`

Concurrencia : Ejercicios



- **Ejercicio 26 (cont.):**

  Si se dispone de **monitores** como herramienta de sincronización, desarrollar el código de estos cuatro procesos y del monitor que utilicen, teniendo en cuenta que ha de cumplirse que:

  - Cuando no hay ningún vehículo atravesando el túnel se permite atravesarlo a cualquiera, sea cual sea su sentido de circulación.
  - Sólo pueden cruzar el túnel varios vehículos de forma simultánea si lo hacen en el mismo sentido.